Mapeo Objeto Relacional (ORM)

Unidad 3

Tabla de contenido

Concepto de Mapeo objeto-relacional 4

Herramientas ORM. Características y herramientas más utilizadas 5
Características 6
Herramientas ORM más utilizadas 7

Introducción a Hibernate 9

Arquitectura de Hibernate 10

Instalación y configuración de Hibernate 11
NetBeans 11
Visual Studio Code 13
Eclipse 14

Ficheros de configuración y mapeo. Estructura y propiedades 15
A.Ficheros de configuración. Propiedades 16
B.Ficheros de mapeo. Estructura, elementos y propiedades. 22
C.Anotaciones de mapeo 26
Principales anotaciones de mapeo en Hibernate 26
- 1. Anotaciones de mapeo de entidad básica 26
- 2. Anotaciones de mapeo de atributos 28
- 3. Anotaciones de relaciones entre entidades 29
- 4. Anotaciones adicionales de configuración de relaciones 35

Ejemplo completo de mapeo con anotaciones 36

¿Qué es el patrón Singleton? 39

Mapeo de colecciones, relaciones y herencia 42
1. Mapeo de colecciones. 42
2. Mapeo de relaciones o asociaciones. 42
3. Mapeo de herencia. 43
Ejemplo: Mapeo Uno-a-Muchos (List) 45

Clases persistentes 50

Sesiones; estados de un objeto 53

Carga, almacenamiento y modificación de objetos 54
Almacenamiento y modificación de objetos 55

Consultas SQL 58

Lenguajes propios de la herramienta ORM: HQL 59
Sintaxis del lenguaje HQL 60
Sentencias consulta (SELECT) 60
Consultas por criterios 70

Gestión de Transacciones 74

Ejemplo básico de cómo usar Hibernate con Mapeo con Anotaciones 76
Pasos previos 76
Paso 1: Configura Hibernate con hibernate.cfg.xml 77
Paso 2: Crea una Entidad Java (Usuario.java) 78

Paso 3: Crea una Clase de Utilidad para Hibernate (HibernateUtil.java)....79
Paso 4: Operaciones CRUD en una Clase Principal (Main.java)....80
Paso 5: Ejecuta el Proyecto....82

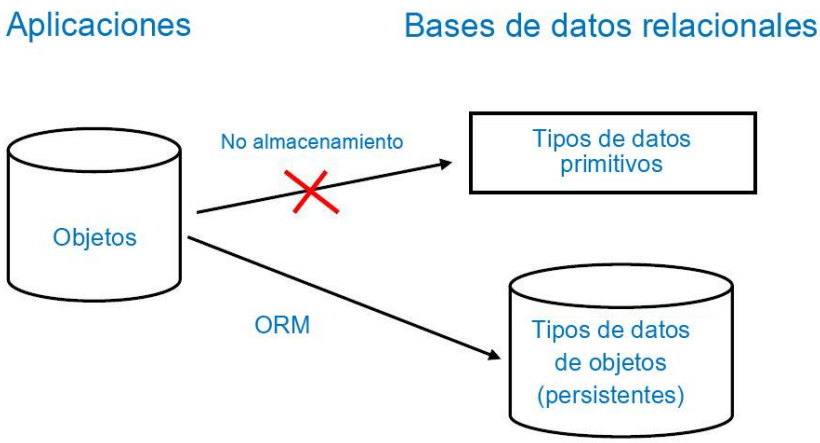
Concepto de Mapeo objeto-relacional

A la hora de almacenar los datos de un programa orientado a objetos en una base de datos relacional, surge un inconveniente debido a incompatibilidad de sistemas de tipos de datos. En el software orientado a objetos, la información se representa como clases y objetos. En las bases de datos relacionales, como tablas y sus restricciones. Por tanto, para almacenar la información tratada en un programa orientado a objetos en una base de datos relacional es necesaria una traducción entre ambas formas.

El mapeo objeto-relacional (ORM) soluciona este problema. Es una técnica de programación que se utiliza con el propósito de convertir datos entre el utilizado en un lenguaje de programación orientado a objetos y el utilizado en una base de datos relacional, gracias a la persistencia *(proceso mediante el cual los objetos en un programa orientado a objetos se almacenan de manera duradera en una base de datos relacional, de forma que puedan ser recuperados y utilizados posteriormente)*. Esto posibilita el uso en las bases de datos relacionales de las características propias de la programación orientada a objetos (básicamente herencia y polimorfismo).

NOTA: La mayoría de las aplicaciones se construyen usando técnicas de programación orientada a objetos; sin embargo, los sistemas de bases de datos más extendidos son de tipo relacional.

Las bases de datos más extendidas son del tipo relacional y estas sólo permiten guardar tipos de datos primitivos (enteros, cadenas de texto, etc.) por lo que no se puede guardar de forma directa los objetos de la aplicación en las tablas. Por tanto, se debe convertir los valores de los objetos en valores simples que puedan ser almacenados en una base de datos (y poder recuperarlos más tarde).



El mapeo objeto-relacional surge, pues, para dar respuesta a esta problemática: traducir los objetos a formas que puedan ser almacenadas en bases de datos preservando las propiedades de los objetos y sus relaciones; estos objetos se dice entonces que son persistentes.

El ORM se encarga, de forma automática, de convertir los objetos y registros y viceversa, simulando así tener una base de datos orientada a objetos.

En el siguiente enlace encontrarás un documento muy interesante sobre la problemática asociada al almacenamiento de objetos y al concepto de mapeo objeto-relacional.

https://informaticaviernes2011.wordpress.com/wp-content/uploads/2011/07/orm_victor_raquel_berni.pdf

Herramientas ORM. Características y herramientas más utilizadas

Las herramientas ORM se utilizan para dar solución al problema de que, en la programación orientada a objetos, la gestión de datos se implementa usando objetos; sin embargo, en los sistemas de gestión de bases de datos SQL sólo se pueden almacenar y manipular valores escalares organizados en tablas relacionales.

El Mapeo Objeto-Relacional (ORM) ó Object-Relational Mapping es una técnica que permite interactuar con bases de datos relacionales de una manera más natural para los programadores orientados a objetos. En lugar de escribir consultas SQL manualmente para realizar operaciones con la base de datos, los programadores pueden manipular las tablas de la base de datos a través de objetos Java. Esto hace que el acceso y la gestión de los datos sea más sencillo y menos propenso a errores.

NOTA: El ORM mapea las clases de Java con las tablas de una base de datos, de manera que:

1. Cada clase en Java corresponde a una tabla en la base de datos.
2. Cada atributo de la clase Java corresponde a una columna de la tabla.
3. Cada instancia de una clase Java corresponde a una fila en la tabla.

Las herramientas ORM pues, actúan como un puente que conecta las ventajas de los RDBMS (Sistemas de Gestión de Bases de Datos Relacionales) con la buena representación de estos en un lenguaje Orientado a Objetos, o, dicho en otras palabras, nos lleva de la base de datos al lenguaje de programación.

Utilizando una herramienta ORM podemos recuperar los objetos de un programa en formas compatibles con el sistema de almacenamiento de los gestores de bases de datos relacionales.

a. No.
b. Si.

Características

- Las herramientas ORM facilitan el mapeo de atributos entre una base de datos relacional y el modelo de objetos de una aplicación, mediante archivos declarativos (XML) que permiten establecer estas relaciones.
- Gracias a las ORM (Mapeo Objeto-Relacional), podemos conectar con una base de datos relacional para extraer la información contenida en objetos de programas que están almacenados. Para ello, sólo tendremos que definir la forma en la que establecer la correspondencia entre las clases y las tablas una sola vez (indicando qué propiedad se corresponde con cada columna, qué clase con cada tabla, etc.).

Una vez hecho esto, podremos utilizar POJOs (Plain Old Java Object) de nuestra aplicación e indicar a la ORM que los haga persistentes, consiguiendo que una sola herramienta pueda leer o escribir en la base de datos utilizando VOs (Value Object) directamente. Esto significa que no es necesario escribir consultas SQL manuales para estas operaciones.

NOTA: POJOs (Plain Old Java Object), son objetos Java simples que representan entidades de la base de datos. En el contexto de Hibernate, un POJO suele mapear una tabla de la base de datos y define los atributos como propiedades de la clase.

NOTA: VOs (Value Object), los Value Objects son objetos que contienen los datos que queremos manipular y transportar en la aplicación, representando el estado de una entidad en un momento dado.

Herramientas ORM más utilizadas



Entre las herramientas ORM más relevantes encontramos las siguientes:

- **Java Persistence Api (JPA):**

El Java Persistence API (JPA) es una especificación de Sun Microsystems para la persistencia de objetos Java a cualquier base de datos relacional. Esta API fue desarrollada para la plataforma JEE *(Java Platform Enterprise Edition, ahora conocida como Jakarta EE, es un conjunto de especificaciones, herramientas y APIs que facilitan el desarrollo de aplicaciones empresariales robustas, seguras, y escalables en Java. Está diseñada para sistemas distribuidos, aplicaciones web, servicios RESTful y componentes que requieren transacciones, concurrencia y seguridad.)* e incluida en el estándar de EJB 3.0, formando parte de la Java Specification Request JSR 220.

Para su utilización, JPA requiere de J2SE 1.5 (también conocida como Java 5) o superior, ya que hace uso intensivo de las nuevas características de lenguaje Java, como las anotaciones y los genéricos.

- **Hibernate:**

Uno de los frameworks ORM más populares para Java. Es una herramienta de Mapeo objeto-relacional (ORM) para la plataforma Java y disponible también para .Net con el nombre de NHibernate que facilita el mapeo de atributos entre una base de datos relacional tradicional y el modelo de objetos de una aplicación. Utiliza archivos declarativos (XML) o anotaciones en los beans de las entidades que permiten establecer estas relaciones.

Es la implementación más popular de **JPA (Java Persistence API)** y ofrece una completa solución de mapeo objeto-relacional. **Hibernate** gestiona automáticamente las conversiones entre clases Java y tablas de bases de datos.

Hibernate tiene su propio lenguaje de programación llamado HQL (Hibernate Query Language), una especie de SQL reducido. Además Hibernate es software libre, distribuido bajo los términos de la licencia GNU LGPL.



- **EclipseLink**

Es la implementación de referencia para JPA. Desarrollado por Oracle, proporciona una solución robusta y escalable para aplicaciones empresariales. Características:

✓ Soporte para mapeo avanzado de datos.
✓ Compatible con múltiples tipos de bases de datos.
✓ Integración con tecnologías como JAXB (para trabajar con XML).

- **iBatis/MyBatis:**

iBatis es un framework de persistencia desarrollado por la Apache software Foundation. Al igual que que el resto de los proyectos desarrollados por la ASF, iBatis es una herramienta de código libre.

iBatis sigue el mismo esquema de uso que Hibernate; se apoya en ficheros de mapeo XML para persistir la información contenida en los objetos en un repositorio relacional.



iBatis se transformó posteriormente en **MyBatis**, que es la versión más actualizada y mantenida del framework. MyBatis incluye mejoras significativas sobre iBatis y es la opción recomendada si estás empezando un nuevo proyecto.

Diferencias entre iBatis (MyBatis) y un ORM tradicional como Hibernate:

- iBatis/MyBatis: El desarrollador tiene control total sobre el SQL y el mapeo de los resultados a objetos Java, lo que es ideal para escenarios donde se necesita SQL específico y optimizado. Sin embargo, no abstrae completamente la base de datos.
- Hibernate: El framework genera automáticamente el SQL en función de las entidades y relaciones entre ellas. Proporciona una abstracción de alto nivel y gestiona gran parte de las interacciones con la base de datos, lo que simplifica el desarrollo.

La herramienta ORM Hibernate se caracteriza por:

a) No ser compatible con la plataforma .Net.
b) Ser software libre.
c) Ser una especificación de Sun Microsystems.

Relaciona cada herramienta con su desarrollador

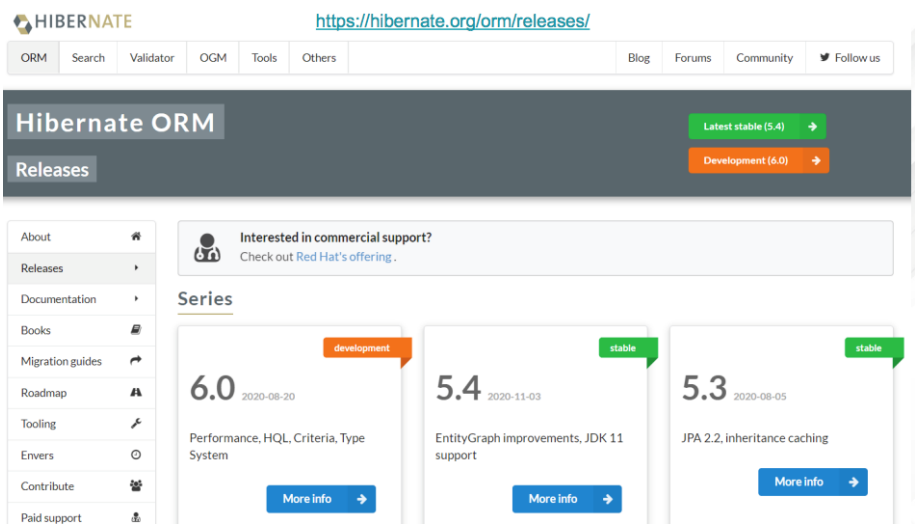
Tipo de Herramienta	Desarrollador
a. Hibernate.	1. Sun Microsystems.
b. JPA.	2. Apache software Foundation.
c. iBatis.	3. Oracle.

Introducción a Hibernate

- Hibernate es una **herramienta (framework) de Java** que facilita el trabajo con bases de datos relacionales desde aplicaciones orientadas a objetos. Su objetivo principal es evitar que el programador tenga que escribir SQL manualmente para realizar operaciones comunes sobre datos.
- Hibernate permite **mapear** clases de Java (objetos) a tablas de una base de datos relacional. Este proceso de correspondencia entre:
  clases ↔ tablas
  atributos ↔ columnas
  relaciones entre objetos ↔ relaciones entre tablas
  Se conoce como **mapeo** y puede realizarse mediante:
  1. **anotaciones** (forma más moderna)
  2. **ficheros XML** (forma clásica)
- Respecto a la **persistencia de objetos**, Hibernate permite que los objetos de una aplicación Java **se almacenen y recuperen automáticamente** en una base de datos relacional. El programador trabaja con objetos, Hibernate se encarga de traducir esas operaciones a SQL.
- Hibernate ofrece su propio **lenguaje de consultas**: **HQL**, que es similar a SQL pero está orientado a **entidades** en lugar de tablas. Permite realizar consultas de forma más natural dentro de un programa Java
  Ejemplo:
  FROM Producto p WHERE p.precio > 1000
- Para conectar una aplicación Java con la base de datos, Hibernate necesita:
  1. un **archivo de configuración (hibernate.cfg.xml)**
  2. **clases mapeadas** (con **anotaciones** o XML)

Este conjunto de definiciones permite que Hibernate "entienda" cómo relacionar el programa con la base de datos real.

Para ver info de Hibernate ver, https://hibernate.org/orm/releases/

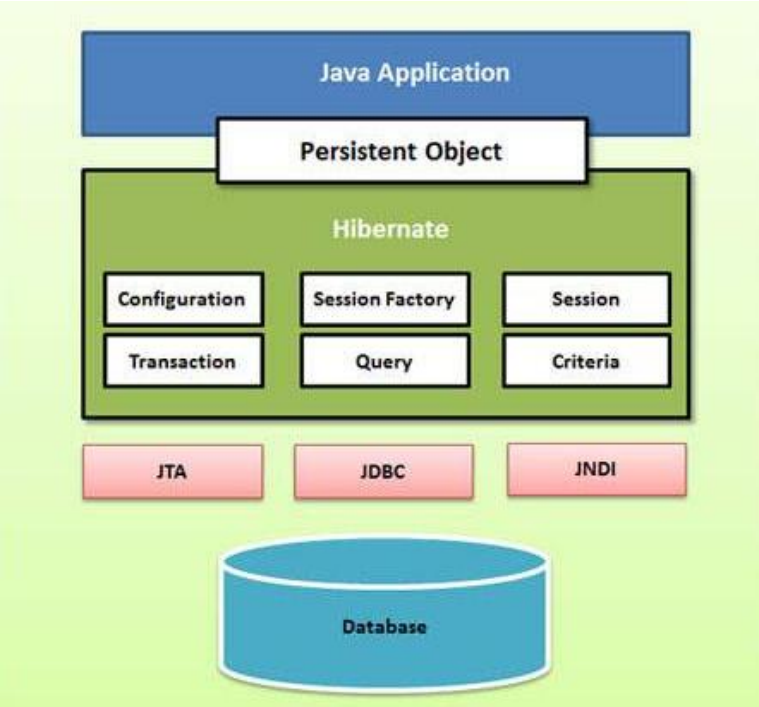


Arquitectura de Hibernate

La arquitectura de Hibernate es en capas para mantenerlo aislado de tener que conocer las API subyacentes. Hibernate usa los datos de la base de datos y proporciona **persistencia** entre los diferente objetos.
A continuación algunos de los **interfaces** que presenta Hibernate:

- Configuration: Se configura hibernate, se especifica la ubicación de los documentos que son utilizados para el mapeo de objetos. Es decir, carga los archivos de configuración y mapeo de clases, definiendo cómo Hibernate se conectará a la base de datos y cómo se mapearán las entidades Java a las tablas SQL. Es el primer paso para configurar Hibernate.
- SessionFactory: A partir de la configuración, SessionFactory es responsable de crear instancias de Session. Es un objeto pesado que se configura una sola vez y se comparte entre todas las sesiones.
- Session: La Session es una instancia que permite abrir una conexión física a la base de datos. Permite realizar operaciones CRUD (Crear, Leer, Actualizar y Borrar) y también es responsable de gestionar el ciclo de vida de las transacciones.
- Transaction: Representa una unidad de trabajo con la base de datos y la mayor parte del RDBMS soporta la funcionalidad de transacción. Las transacciones en hibernación son manejados por un gestor de transacciones subyacente y de transacción (de JDBC o JTA). Cualquier error en la transacción producirá un fallo de la misma.
- Query: Mediante Session, se pueden crear objetos Query para ejecutar consultas en la base de datos utilizando **HQL** (Hibernate Query Language) o SQL nativo.

De forma gráfica esto se puede ver en la siguiente imagen que se muestra a continuación:



Instalación y configuración de Hibernate

NetBeans

La instalación de Hibernate sobre el IDE NetBeans requiere tener instalado previamente este entorno de desarrollo, junto al JDK. El proceso que se va a desarrollar a continuación se hace con NetBeans 7.2 aunque para la versión 8.2 el proceso es idéntico.

Proceso de instalación de Hibernate:

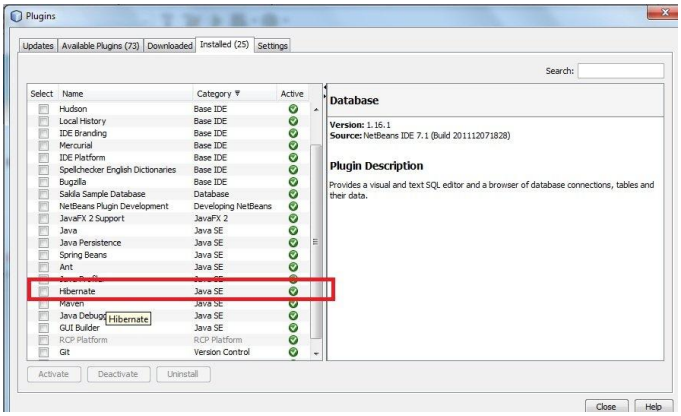
1. Descargar las librerías de Hibernate
   - Descarga Hibernate: Ve al sitio oficial de Hibernate: https://hibernate.org/orm/. Descarga la última versión del paquete Hibernate en formato ZIP.
   - Librerías necesarias: Dentro del ZIP, ubica las librerías básicas:
     - hibernate-core-<version>.jar
     - Dependencias adicionales, como: antlr, dom4j, javassist, javax.persistence, entre otras.
2. Crear un proyecto en NetBeans
3. Agregar Hibernate al proyecto:
   - Proyecto sin Maven (Java SE tradicional):
     - ✓ Haz clic derecho sobre el proyecto y selecciona: Properties > Libraries.
     - ✓ En la pestaña Compile, selecciona Add JAR/Folder y agrega las librerías descargadas de Hibernate.
   - Proyecto con Maven:
     - ✓ Abre el archivo pom.xml.
     - ✓ Agrega las dependencias de Hibernate al bloque <dependencies>. Asegúrate de ajustar las versiones según las más recientes.

```
<dependencies>
    <!-- Hibernate Core -->
    <dependency>
      <groupId>org.hibernate</groupId>
      <artifactId>hibernate-core</artifactId>
      <version>5.6.5.Final</version>
    </dependency>

    <!-- MySQL Connector -->
    <dependency>
      <groupId>mysql</groupId>
      <artifactId>mysql-connector-java</artifactId>
      <version>8.0.28</version>
    </dependency>
</dependencies>
```

Se destaca que el IDE NetBeans 8.X ya tiene incluida la versión 4.X de Hibernate, si deseas una versión superior debes descargarla según el enlace dado.

Una vez que inicializamos NetBeans, podemos encontrar la herramienta Hibernate como un plugin de este entorno de desarrollo. Lo único que tienes que hacer es ir a la lista de plugins de NetBeans y comprobar que lo tienes instalado; en caso contrario, deberás instalarlo desde la opción de plugins disponibles.



Captura de pantalla donde se muestra la lista de plugins instalados en NetBeans y, dentro de un rectángulo de borde rojo, se observa el plugin de Hibernate. Para utilizar Hibernate en una aplicación, es necesario conocer cómo configurarlo.

Hibernate puede configurarse y ejecutarse en la mayoría de aplicaciones Java y entornos de desarrollo. El archivo de configuración de Hibernate recibe el nombre de **Hibernate.cfg.xml** y contiene información sobre la conexión de la base de datos y otras propiedades. Al crearlo, hay que especificar la conexión a la base de datos.

Visual Studio Code

La instalación de Hibernate sobre el IDE Visual Studio Code requiere tener instalado previamente este entorno de desarrollo, junto al JDK, preferiblemente la versión 8 o superior.
Pasos:

1. Configurar las Librerías de Hibernate: Descarga los archivos .jar de Hibernate desde el sitio oficial de Hibernate o desde Maven Central. Los archivos necesarios incluyen hibernate-core y otras dependencias como javax.persistence y el driver JDBC para conectar a la base de datos (por ejemplo, MySQL Connector para MySQL).
2. Configurar el Classpath: Agrega los archivos .jar de Hibernate y el driver JDBC al classpath de tu proyecto en Visual Studio Code.
3. Configurar Maven o Gradle (opcional): Para facilitar la gestión de dependencias, puedes configurar Maven o Gradle en tu proyecto, lo que permitirá descargar y manejar automáticamente las librerías necesarias de Hibernate.

```
<dependencies>
    <!-- Hibernate Core -->
    <dependency>
      <groupId>org.hibernate</groupId>
      <artifactId>hibernate-core</artifactId>
      <version>5.6.5.Final</version>
    </dependency>

    <!-- MySQL Connector -->
    <dependency>
      <groupId>mysql</groupId>
      <artifactId>mysql-connector-java</artifactId>
      <version>8.0.28</version>
    </dependency>
</dependencies>
```

Para utilizar Hibernate en una aplicación, es necesario conocer cómo configurarlo.

Eclipse

La instalación de Hibernate sobre el IDE Eclipse requiere tener instalado previamente este entorno de desarrollo, junto al JDK, preferiblemente la versión 8 o superior.
Pasos:

1. Crear un Nuevo Proyecto Java en Eclipse

Abre Eclipse y crea un nuevo Proyecto Java:
Ve a File > New > Java Project.
Asigna un nombre al proyecto (por ejemplo, HibernateExample) y finaliza la creación.

2. Descargar las Librerías de Hibernate

Descarga Hibernate desde el sitio oficial de Hibernate o, si prefieres, configúralo con Maven o Gradle (opcional, pero recomendado).

Si no usas Maven o Gradle, asegúrate de descargar las siguientes librerías:

- ✓ hibernate-core
- ✓ javax.persistence (para anotaciones JPA)
- ✓ Driver JDBC para la base de datos (por ejemplo, MySQL Connector si usas MySQL).

Opción Maven/Gradle: Si usas Maven o Gradle, puedes saltar el paso de agregar manualmente los .jar y, en cambio, agregar las dependencias en el archivo de configuración pom.xml o build.gradle.

3. Agregar las Librerías al Proyecto en Eclipse

Crea una carpeta llamada lib en la raíz del proyecto.
Copia los archivos .jar descargados (librerías de Hibernate y el driver JDBC) a esta carpeta.
Haz clic derecho en cada archivo .jar en lib, selecciona Build Path > Add to Build Path.

Para utilizar Hibernate en una aplicación, es necesario conocer cómo configurarlo.

En los siguientes puntos del tema veremos más detenidamente el proceso de configuración.

Ficheros de configuración y mapeo. Estructura y propiedades

Para empezar a trabajar con Hibernate es necesario configurar la herramienta para que conozca qué objetos debe recuperar de la base de datos relacional y en qué lugar los hará persistir. Por tanto, el primer paso será tener una base de datos relacional con la que poder trabajar.

En NetBeans, por ejemplo, cuando se crea el archivo de configuración de Hibernate usando el asistente, podemos especificar la conexión a la base de datos, eligiendo de una lista de conexiones de bases de datos registradas en el IDE. Cuando se genera el archivo de configuración, el IDE añade de forma automática detalles de la conexión e información basada en la conexión de la base de datos seleccionada. El IDE añade también las bibliotecas de Hibernate en el proyecto. Después de crear el fichero de configuración, éste puede ser editado usando el editor interactivo, o editar directamente el código XML.

El fichero de configuración contiene información sobre la base de datos a la que vamos a conectar la aplicación. Si la aplicación se va a conectar a varias bases de datos, sería necesario definir tantos archivos de configuración como bases de datos a las que nos queramos conectar.

Para tener toda esta información, en Hibernate surgen dos ficheros distintos:

- El archivo de propiedades de Hibernate (Hibernate.properties), que es el encargado de determinar todos los aspectos relacionados con el gestor de la base de datos y las conexiones con él.
- Los archivos que definen el emparejamiento (mapping) de propiedades con tablas y columnas (*.hbm.xml). Recordad que se pueden usar para el mapeo estos ficheros XML o **Anotaciones** (en este curso trabajaremos con anotaciones, que explicaremos en apartados posteriores).

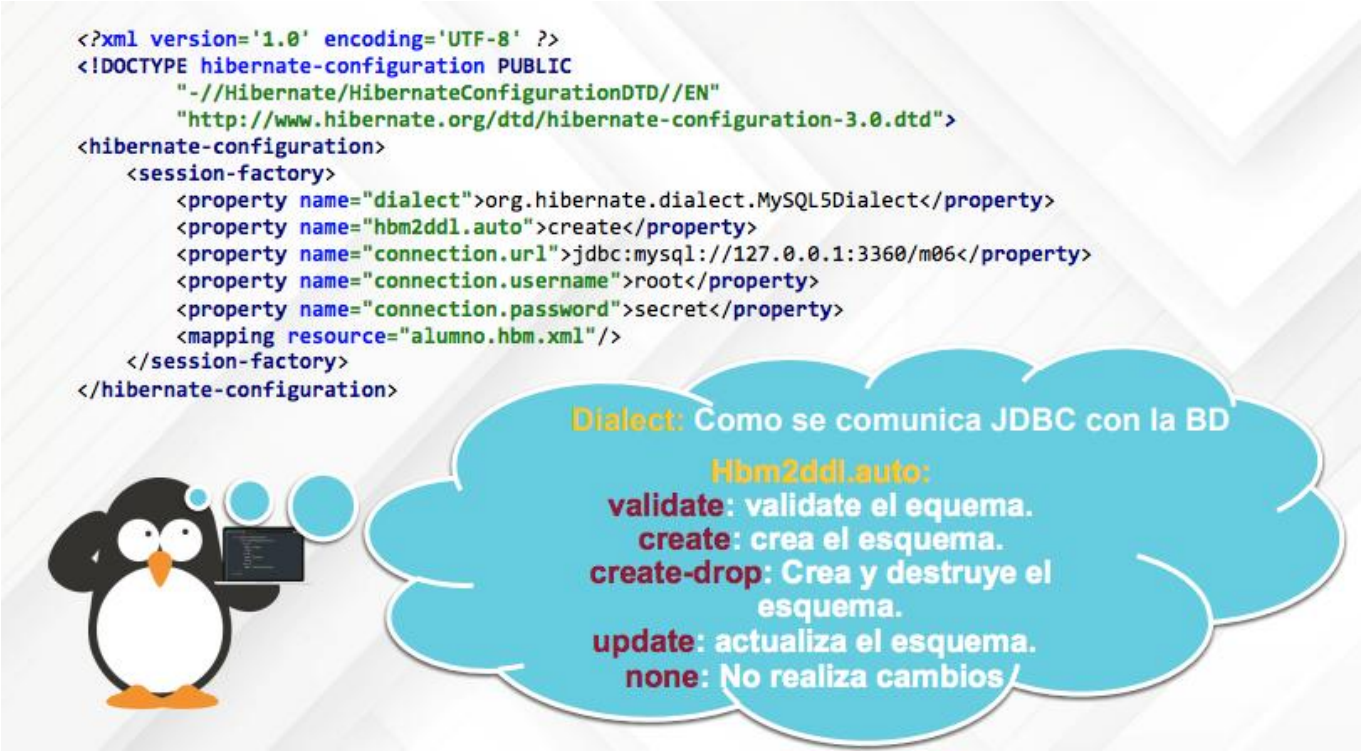
Para utilizar **Hibernate** necesitamos tener una base de datos relacional.

En el siguiente enlace encontrarás un documento donde se expone con más detalle aspectos básicos de configuración de Hibernate. https://howtodoinjava.com/hibernate-tutorials/

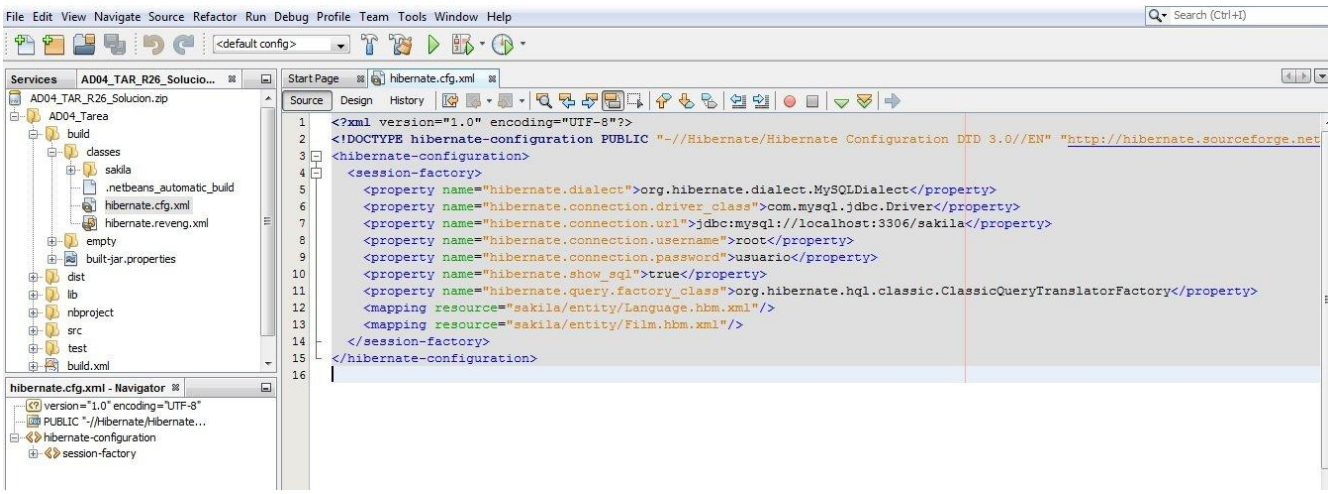
A.Ficheros de configuración. Propiedades

1. Hibernate.cfg.xml

El archivo de configuración de Hibernate es el **Hibernate.cfg.xml** y contiene información sobre la conexión de base de datos, las asignaciones de recursos y otras propiedades de conexión.



Al seleccionar una conexión a una base de datos en NetBeans, se crea un archivo de configuración en Hibernate que guarda los detalles de esa conexión. NetBeans también añade de forma automática la biblioteca de Hibernate para la ruta de clase que tendrá nuestro proyecto. Después de la creación del archivo **Hibernate.cfg.xml**, podemos editarlo o modificar el código xml mediante el editor xml.



Captura de pantalla donde se muestra la apariencia del archivo de configuración de Hibernate en NetBeans.

Las propiedades más importantes del fichero Hibernate.cfg.xml son:
- ✓ Hibernate.dialect: Dialecto o lenguaje empleado. Por ejemplo, MySQL.
- ✓ Hibernate.connection.driver_class. Driver utilizado para la conexión con la base de datos.
- ✓ Hibernate.connection.url. Dirección de la base de datos con la que se va a conectar Hibernate.
- ✓ Hibernate.connection.username. Nombre del usuario que va a realizar la extracción de información. Por defecto, el nombre de usuario es root.
- ✓ Hibernate.connection.password. Contraseña el root.
- ✓ Hibernate.show_sql. Para mostrar la herramienta. Por defecto, su valor es true.

Finalmente una vez mapeadas las clases/tablas ya sea en formato fichero o con anotaciones deberemos añadir el mapeo al fichero hibernate.cfg.xml de la siguiente forma:
- ✓ Mapeo con fichero:
  <mapping resource="persona.hbm.xml"/>
- ✓ Mapeo con anotaciones:
  <mapping class="com.prueba.entities.Persona"/>

El archivo Hibernate.cfg.xml contiene información sobre:
a) La conexión, recursos y otras propiedades como nombre de usuario, contraseña, etc.
b) La conexión a la base de datos.
c) La dirección de la base de datos y la versión del IDE utilizada.

2. Hibernate.reveng.xml

Otro fichero importante es **hibernate.reveng.xml**, es un archivo de **ingeniería inversa** en Hibernate. Define cómo Hibernate debe interpretar una base de datos existente y especificar qué tablas y columnas incluir, ignorar o modificar en el proceso de generación de entidades y mapeos de Java. En esencia, permite que Hibernate "lea" una base de datos y automáticamente genere el código necesario para trabajar con ella.

Funcionalidades de hibernate.reveng.xml:
- Seleccionar tablas: Puedes especificar qué tablas de la base de datos deben ser mapeadas en entidades Java.
- Filtrar columnas: Permite omitir o personalizar ciertas columnas, por ejemplo, si deseas excluir una columna que no debe ser parte de tu modelo de datos.
- Configurar relaciones: Define relaciones entre tablas, como asociaciones One-to-Many o Many-to-One.
- Modificar nombres: Puedes personalizar los nombres de clases y propiedades generados para que sean diferentes a los nombres en la base de datos.

Su estructura es la siguiente:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-reverse-engineering SYSTEM
      "http://hibernate.sourceforge.net/hibernate-reverse-engineering-3.0.dtd">
<hibernate-reverse-engineering>
   <table-filter match-name="usuarios" match-schema="public"/>
   <table name="usuarios" class="com.example.Usuario">
      <column name="id_usuario" property="id" type="long"/>
      <column name="nombre_usuario" property="nombre" type="string"/>
   </table>
</hibernate-reverse-engineering>
```

- <table-filter match-name="usuarios" match-schema="public"/>: Filtra para incluir solo la tabla usuarios del esquema public.
- <table name="usuarios" class="com.example.Usuario">: Indica que la tabla usuarios se mapeará a la clase Usuario en el paquete com.example.
- <column name="id_usuario" property="id" type="long"/>: Define que la columna id_usuario en la base de datos se mapeará como la propiedad id en la clase generada, de tipo long.
- <column name="nombre_usuario" property="nombre" type="string"/>: Mapea nombre_usuario a nombre en la clase Java como un String.

¿Para qué se usa?

El archivo hibernate.reveng.xml es especialmente útil cuando:

1. Tienes una base de datos existente y deseas **generar automáticamente el código Java** para interactuar con sus tablas.
2. Quieres **evitar escribir manualmente clases de entidades y mapeos**.
3. Necesitas un proceso rápido de generación de código basado en los esquemas de base de datos que pueden cambiar con frecuencia.

¿Cómo usarlo en Eclipse con Hibernate Tools?

1. **Instala Hibernate Tools** en Eclipse.
2. Crea un archivo hibernate.reveng.xml en la carpeta src o en src/main/resources.
3. Configura las tablas y columnas que deseas mapear en hibernate.reveng.xml.
4. Usa la opción de "**Hibernate Code Generation**" de Hibernate Tools para generar las entidades automáticamente.

El archivo hibernate.reveng.xml es una herramienta muy útil para acelerar el desarrollo en aplicaciones que necesitan conectarse a bases de datos preexistentes, permitiendo que Hibernate genere automáticamente los mapeos necesarios.

¿Cómo usarlo en Visual Studio Code?

En Visual Studio Code, puedes utilizar hibernate.reveng.xml para realizar ingeniería inversa con Hibernate y generar clases de entidades a partir de una base de datos existente. Aunque VS Code no tiene soporte integrado para Hibernate Tools como Eclipse, puedes lograr el mismo resultado configurando Hibernate con Maven o Gradle y usando algunos comandos en la terminal. A continuación te explico cómo hacerlo:

1. Configurar el Proyecto en Visual Studio Code: Primero, crea un proyecto de Java en Visual Studio Code y configúralo con Maven o Gradle para manejar las dependencias de Hibernate.
2. Configurar las Dependencias de Hibernate en pom.xml (si usas Maven). Asegúrate de incluir las dependencias de Hibernate y de Hibernate Tools en el archivo pom.xml:

```
<dependencies>
    <!-- Hibernate core -->
    <dependency>
        <groupId>org.hibernate</groupId>
        <artifactId>hibernate-core</artifactId>
        <version>5.6.3.Final</version>
    </dependency>

    <!-- Hibernate tools for reverse engineering -->
    <dependency>
        <groupId>org.hibernate</groupId>
        <artifactId>hibernate-tools</artifactId>
        <version>5.4.0.Final</version>
    </dependency>

    <!-- MySQL connector (si usas MySQL) -->
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.27</version>
    </dependency>
</dependencies>
```

Nota: Ajusta las versiones según sea necesario.

3. Crear el Archivo hibernate.reveng.xml: En la carpeta src/main/resources de tu proyecto, crea el archivo hibernate.reveng.xml para definir las tablas y columnas que deseas mapear desde la base de datos. Aquí tienes un ejemplo:

```
<!DOCTYPE hibernate-reverse-engineering SYSTEM "http://hibernate.sourceforge.net/hibernate-reverse-engineering-3.0.dtd">
<hibernate-reverse-engineering>
  <table-filter match-name="usuarios" match-schema="public"/>
  <table name="usuarios" class="com.example.Usuario">
    <column name="id_usuario" property="id" type="long"/>
    <column name="nombre_usuario" property="nombre" type="string"/>
  </table>
</hibernate-reverse-engineering>
```

Este archivo le indica a Hibernate que realice ingeniería inversa solo en la tabla usuarios y que mapee las columnas especificadas a las propiedades de la clase Usuario.

4. Crear el Archivo de Configuración hibernate.cfg.xml: También en src/main/resources, crea el archivo hibernate.cfg.xml para definir la configuración de Hibernate y la conexión a la base de datos.

```
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
    <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/tu_base_datos</property>
    <property name="hibernate.connection.username">tu_usuario</property>
    <property name="hibernate.connection.password">tu_contraseña</property>
    <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
    <property name="hibernate.hbm2ddl.auto">none</property>
  </session-factory>
</hibernate-configuration>
```

Nota: Cambia tu_base_datos, tu_usuario, y tu_contraseña según tu configuración.

5. Ejecutar la Ingeniería Inversa con Hibernate Tools (En Terminal): Para usar hibernate.reveng.xml en VS Code, puedes ejecutar Hibernate Tools desde la terminal con Maven. Añade el siguiente plugin de Maven en tu archivo pom.xml para facilitar la ejecución de ingeniería inversa:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.hibernate.orm.tooling</groupId>
      <artifactId>hibernate-tools-maven-plugin</artifactId>
      <version>5.4.0.Final</version>
      <executions>
        <execution>
          <goals>
            <goal>hbm2java</goal>
          </goals>
        </execution>
      </executions>
      <configuration>
        <revengFile>src/main/resources/hibernate.reveng.xml</revengFile>
        <configurationFile>src/main/resources/hibernate.cfg.xml</configurationFile>
        <outputDirectory>src/main/java</outputDirectory>
      </configuration>
    </plugin>
  </plugins>
</build>
```

https://central.sonatype.com/artifact/org.hibernate/hibernate-tools-maven-plugin/versions

6. Ejecutar el Plugin de Hibernate en la Terminal: En la terminal de Visual Studio Code, ejecuta el siguiente comando para generar las entidades a partir de la base de datos:

```
mvn hibernate-tools:hbm2java
```

Este comando usará hibernate.reveng.xml y hibernate.cfg.xml para conectarse a la base de datos, realizar la ingeniería inversa y generar las clases de entidad en el directorio src/main/java.

7. Verificar los Archivos Generados: Después de ejecutar el comando, ve a la carpeta src/main/java y verifica que se hayan generado las clases de entidad correspondientes a las tablas especificadas en hibernate.reveng.xml.

B.Ficheros de mapeo. Estructura, elementos y propiedades.

1. Fichero de mapeo ".hbm.xml"

Originalmente Hibernate utilizaba unos ficheros de mapeo de archivos con la extensión .hbm.xml (Hibernate Mapping XML). El propósito principal de un archivo .hbm.xml es especificar cómo las entidades de una aplicación Java se mapean a las tablas y columnas de una base de datos relacional. Define:

- Las tablas correspondientes a cada clase.
- Las columnas que se mapean a los atributos de cada clase.
- Las relaciones entre las clases (como @OneToMany, @ManyToOne, etc.).
- Configuraciones adicionales como estrategias de generación de claves, opciones de carga (lazy o eager), y más.

Para cada clase que queremos persistir se creará un fichero xml con la información que permitirá mapear la clase a una base de datos relacional. Este fichero estará en el mismo paquete que la clase a persistir.

```
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="Persistente.Departamentos" table="DEPARTAMENTOS" schema="ANTONIO">
    <id name="deptNo" type="byte">
      <column name="DEPT_NO" precision="2" scale="0" />
      <generator class="assigned" />
    </id>
    <property name="dnombre" type="string">
       <column name="DNOMBRE" length="15" />
    </property>
    <property name="loc" type="string">
      <column name="LOC" length="15" />
   </property>
    <set name="empleadoses" inverse="true">
    <key>
      <column name="DEPT_NO" precision="2" scale="0" not-null="true" />
    </key>
     <one-to-many class="Persistente.Empleados" />
    </set>
    </class>
</hibernate-mapping>
```

Un archivo .hbm.xml generalmente incluye:

Elemento raíz <hibernate-mapping>: Este es el elemento contenedor, todos los ficheros de mapeo comienzan y acaban con esta etiqueta.

Elemento <class>: Mapea una clase de Java a una tabla de la base de datos. Contiene:

- name: El nombre de la clase de Java.
- table: El nombre de la tabla de la base de datos.
- Dentro del <class>, se definen el identificador (clave primaria) y las propiedades.

Elemento <id>: Define el atributo que será la clave primaria en la tabla.

- name: Nombre del atributo en la clase de Java.
- column: Columna correspondiente en la tabla.
- generator: Estrategia de generación de la clave primaria (por ejemplo, increment, identity, etc.).

El generator en el id es como se va a crear esta PK en la base de datos. Existen diferentes tipos:

- ✓ AUTO→Establecerá un valor basado en el tipo del atributo que puede ser numérico o UUID (Universally Unique Identifier), para valores numéricos utilizará una secuencia, para valores UUID utilizará un UUIDGenerator.
- ✓ UUID→Genera un identificador único universal (UUID) para la PK, que es una cadena alfanumérica globalmente única. Útil en aplicaciones distribuidas donde se necesita un identificador único, independientemente de la base de datos. El valor es único en diferentes bases de datos y entornos.
- ✓ ASSIGNED→Te da control total sobre cómo se asignan los valores de las claves primarias. No es común en aplicaciones modernas, ya que suele ser preferible delegar la generación de claves primarias al sistema de bases de datos (usando generadores como native, increment, o identity)
- ✓ IDENTITY→Quiere decir que el id será AUTO_INCREMENT.
- ✓ SEQUENCE → Seguirá una secuencia que se debe proporcionar, es similar al AUTO_INCREMENT pero nos permite configurar la secuencia.
- ✓ INCREMENT→Hibernate calcula el valor de PK incrementándolo en uno sobre el último valor almacenado. Solo recomendable en aplicaciones de una sola instancia, ya que el uso concurrente en varias instancias puede causar conflictos de PK. Útil en entornos pequeños donde la base de datos no soporta secuencias ni auto-incremento.
- ✓ TABLE→Utiliza una tabla oculta que contiene valores de identificadores y se combinan con el nombre de la tabla, es el más ineficiente de todos.

Elemento <property>: Define cada atributo de la clase que se mapea a una columna en la tabla.

- name: Nombre del atributo en la clase de Java.
- column: Columna correspondiente en la base de datos.
- Atributos adicionales, como tipo de datos y si es not-null, etc.

Elementos de relación (<many-to-one>, <one-to-many>, <many-to-many>, <one-to-one>): Especifican las relaciones entre las entidades.

Ejemplo:

```
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
    <!-- Mapea la clase Empleado a la tabla empleados -->
    <class name="com.example.Empleado" table="empleados">
        <!-- Mapea el campo id como clave primaria de la tabla -->
        <id name="id" column="empleado_id">
            <generator class="increment"/>
        </id>
        <!-- Mapea el campo nombre a la columna nombre en la tabla -->
        <property name="nombre" column="nombre" type="string" not-null="true"/>
        <!-- Mapea el campo salario a la columna salario -->
        <property name="salario" column="salario" type="double"/>
        <!-- Ejemplo de relación muchos-a-uno con la tabla departamentos -->
        <many-to-one name="departamento" class="com.example.Departamento" column="departamento_id" not-null="true"/>
    </class>
</hibernate-mapping>
```

Ventajas y Desventajas de los archivos .hbm.xml

- Ventajas:
  - ✓ Separación del código y configuración: Los archivos .hbm.xml permiten mantener las definiciones de mapeo fuera del código Java.
  - ✓ Modificación sin recompilación: Cambios en el mapeo no requieren recompilar el código Java.
  - ✓ Control sobre el mapeo: Permiten definir mapeos detallados y personalizados.

- Desventajas:
  - ✓ Mayor complejidad: Configurar mapeos con XML puede ser más complejo y verboso en comparación con el uso de anotaciones.
  - ✓ Obsoleto: Las anotaciones en las entidades son ahora el método preferido, ya que son más fáciles de leer y mantener, y se integran de forma natural con el código Java.

Uso actual de los archivos .hbm.xml

Aunque el uso de archivos .hbm.xml ha disminuido con la adopción de anotaciones, sigue siendo útil en situaciones donde:

- Se necesita mantener el mapeo separado del código de la aplicación (por ejemplo, en proyectos con alta modularidad).

- Existen configuraciones complejas de mapeo que requieren la flexibilidad de XML.

Los archivos .hbm.xml son una de las primeras metodologías de mapeo en Hibernate y siguen disponibles, aunque muchos proyectos modernos usan anotaciones debido a su simplicidad y facilidad de mantenimiento.

Los ficheros de mapeo se utilizan para relacionar tablas y clases que deseamos extraer de la base de datos que estamos consultando.

a) Falso.
b) Verdadero.

C.Anotaciones de mapeo

Una vez acostumbrados al uso de ficheros de mapeo, es muy sencillo crear nuestra primera aplicación pero tiene ciertos inconvenientes:

- ✓ Si se modifica la clase debemos modificar el fichero (doble trabajo)
- ✓ Se deben definir de forma correcta todos los atributos
- ✓ Es fácil dejarse un atributo sin mapear (dará error)
- ✓ Si la aplicación va creciendo mucho es cada vez más complejo mantener los ficheros.

Es por ello que hibernate se actualizó, **eliminando los ficheros de mapeo y utilizando anotaciones**.

Las anotaciones son unas pequeñas instrucciones que se ponen dentro del código dotando a nuestra clase Entity de la información necesaria para ser mapeada. Las anotaciones más importantes són:

- ✓ @Entity
- ✓ @Table
- ✓ @Id
  - @GeneratedValue
- ✓ @Column

En Hibernate, las anotaciones de mapeo se utilizan para definir cómo las clases de Java y sus propiedades se deben mapear a las tablas y columnas de una base de datos relacional. Estas anotaciones permiten especificar relaciones, claves primarias, claves foráneas, y otras configuraciones de mapeo directamente en el código, eliminando la necesidad de archivos XML separados. Hibernate sigue las especificaciones de JPA (Java Persistence API), por lo que muchas de las anotaciones están definidas en el paquete `javax.persistence`.

Principales anotaciones de mapeo en Hibernate

A continuación se describen las anotaciones más comunes para el mapeo de clases y relaciones en Hibernate.

❖ 1. Anotaciones de mapeo de entidad básica

- @Entity:

Marca una clase Java como una entidad de base de datos, es decir, una tabla.
Es obligatoria en toda clase que queramos persistir con Hibernate. Es decir, debe estar presente en cada clase que se quiera mapear a una tabla de la base de datos.
Ejemplo:

```
@Entity
public class Empleado {
   // ...
}
```

- @Table:

Especifica el nombre de la tabla de base de datos que se va a utilizar para la entidad.
Se usa cuando el nombre de la tabla **no coincide** con el nombre de la clase.
Ejemplo:

```
@Entity
@Table(name = "empleados")
public class Empleado {
    // ...
}
```

- @Id:

Marca un campo como la clave primaria (Primary Key) de la entidad.
Se utiliza junto con generadores de ID, como `@GeneratedValue`.
Ejemplo:

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

- @GeneratedValue:

Define la estrategia de generación de la clave primaria.
Estrategias comunes:

- ✓ **GenerationType.IDENTITY**
  Usa el *auto-increment* de la base de datos (por ejemplo, en MySQL).
- ✓ **GenerationType.SEQUENCE**
  Usa una **secuencia** de la base de datos (Útil en Oracle, PostgreSQL).
- ✓ **GenerationType.TABLE**
  Usa una tabla especial para generar IDs.
- ✓ **GenerationType.AUTO**
  Hibernate selecciona automáticamente la estrategia más adecuada según la BD.

Ejemplo:

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE)
private Long id;
```

❖ 2. Anotaciones de mapeo de atributos

- @Column:

Define el nombre y las propiedades de una columna en la tabla correspondiente.
Se puede usar para especificar el nombre de la columna, el tipo de datos, si es nullable, etc.
Ejemplo:

```
@Column(name = "nombre_empleado", nullable = false, length = 50)
private String nombre;
```

- @Transient:

Indica que un atributo no debe ser persistido en la base de datos.
Útil para campos que solo se utilizan temporalmente en la lógica de la aplicación.
Ejemplo:

```
@Transient
private int edadCalculada;
```

- @Temporal:

Define cómo se debe almacenar un campo de tipo `java.util.Date` en la base de datos.
Los valores pueden ser TemporalType.DATE, TemporalType.TIME, y TemporalType.TIMESTAMP.
Ejemplo:

```
@Temporal(TemporalType.DATE)
private Date fechaNacimiento;
```

- @Lob:

Marca un campo como un Large Object (LOB), útil para almacenar datos grandes como textos o archivos binarios.
Ejemplo:

```
@Lob
private byte[] imagenPerfil;
```

❖ 3. Anotaciones de relaciones entre entidades

Estas anotaciones ayudan a definir relaciones entre diferentes entidades (tablas).

- @OneToOne:

Define una relación uno a uno entre dos entidades.
Un registro de una tabla está relacionado *exactamente con un* registro de otra tabla.
Ejemplo: Cada empleado tiene **un** detalle de información adicional (dirección, etc.).

Clase Empleado

```
@Entity
public class Empleado {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @OneToOne
   @JoinColumn(name = "detalle_id")
   private DetalleEmpleado detalle;
}
```

Clase DetalleEmpleado:

```
@Entity
public class DetalleEmpleado {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   private String direccion;
}
```

En la BD:

Tabla empleado

id (PK)	nombre	detalle_id (FK → detalle_empleado.id)
1	Juan	10
2	Ana	11

Tabla detalle_empleado

id (PK)	direccion
10	Calle 123
11	Av. Central

- @OneToMany:

Define una relación uno a muchos entre una entidad y una colección de otra entidad.
Un registro tiene asociados *varios* registros de otra tabla.
Es necesario usar un contenedor (como `List` o `Set`) en el lado "uno".
Ejemplo: Un empleado puede tener *muchos* proyectos asignados.

Clase Empleado (Lado de Uno)

```
@Entity
public class Empleado {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @OneToMany(mappedBy = "empleado")
   private List<Proyecto> proyectos;
}
```

Clase Proyecto (Lado de Muchos)

```
@Entity
public class Proyecto {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @ManyToOne
   @JoinColumn(name = "empleado_id")
   private Empleado empleado;
}
```

En la BD:

Tabla: empleado

id (PK)	nombre
1	Juan
2	Ana

Tabla: proyecto

id (PK)	nombre	empleado_id (FK → empleado.id)
5	App Web	1
9	API Móvil	1
10	CRM Banco	2

Empleado 1 → tiene 2 proyectos.

- @ManyToOne:

Define una relación muchos a uno.
El campo se mapea como una clave foránea que referencia a otra entidad.
Ejemplo: Varios empleados pertenecen a *un* departamento.

Clase Empleado (Lado de muchos)

```
@Entity
@Table(name = "empleado")
public class Empleado {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    // MANY employees → ONE department
    @ManyToOne
    @JoinColumn(name = "departamento_id") // Clave foránea
    private Departamento departamento;
}
```

Clase Departamento (Lado de uno)

```
@Entity
@Table(name = "departamento")
public class Departamento {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    // Relación inversa (opcional)
    @OneToMany(mappedBy = "departamento")
    private List<Empleado> empleados;

    // Getters y setters
}
```

En la BD:

Tabla: departamento

id (PK)	nombre
2	Recursos Humanos
3	IT

Tabla: empleado

id (PK)	nombre	departamento_id (FK → departamento.id)
1	Juan	3
2	Ana	3
3	Laura	2

Varios empleados comparten el mismo departamento_id.
Departamento 3 → tiene 2 empleados.

- @ManyToMany:

Define una relación muchos a muchos entre dos entidades. Requiere una tabla intermedia en la base de datos para almacenar las claves foráneas de ambas tablas. Ejemplo: Un empleado puede participar en *muchos proyectos* y un proyecto puede tener *muchos empleados*.

Clase Empleado

```
@Entity
public class Empleado {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @ManyToMany
   @JoinTable(
      name = "empleado_proyecto",
      joinColumns = @JoinColumn(name = "empleado_id"),
      inverseJoinColumns = @JoinColumn(name = "proyecto_id")
   )
   private Set<Proyecto> proyectos;
}
```

Clase Proyecto

```
@Entity
public class Proyecto {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @ManyToMany(mappedBy = "proyectos")
   private Set<Empleado> empleados;
}
```

En la BD:

Tabla: empleado

id (PK)	nombre
1	Juan
2	Ana
3	Pedro

Tabla: proyecto

id (PK)	nombre
5	App Web
8	CRM Comercial
9	API Jurídica

Tabla intermedia: empleado_proyecto

empleado_id (FK → empleado.id)	proyecto_id (FK → proyecto.id)
1	5
1	8
2	5
3	9

Cada fila representa una relación entre un empleado y un proyecto.

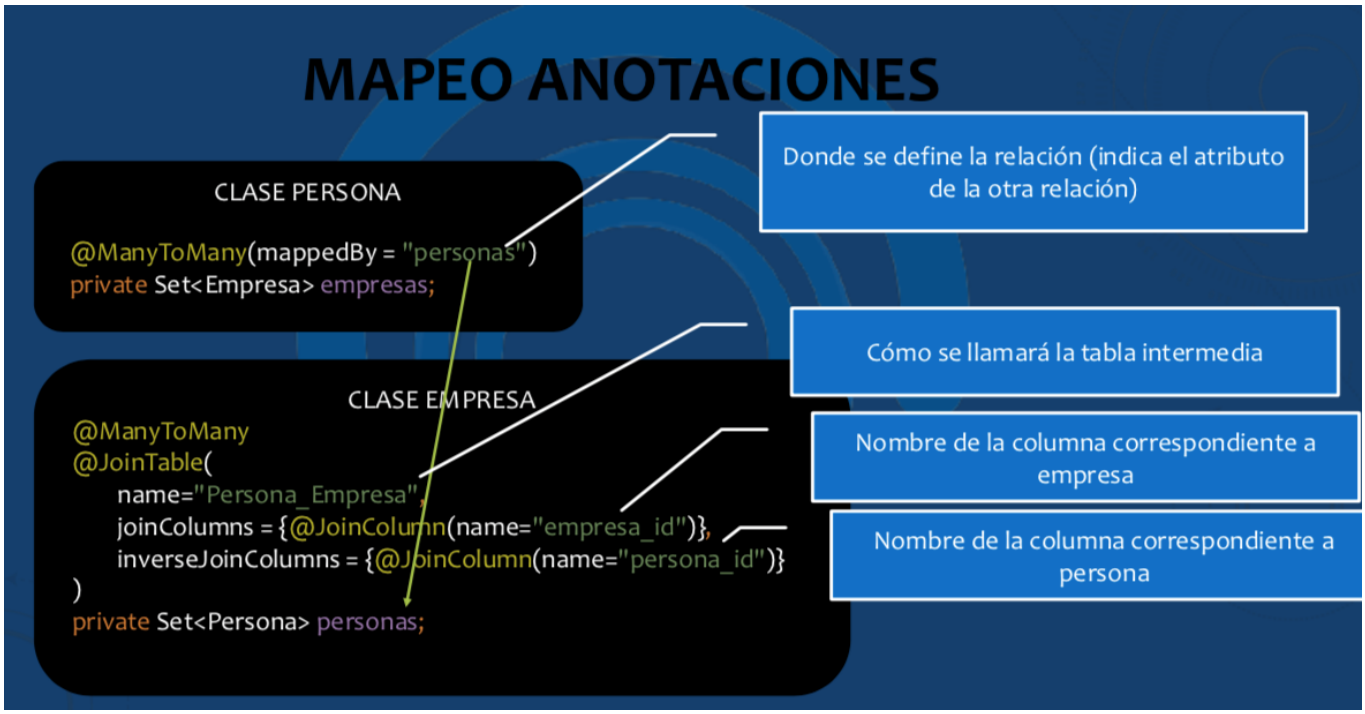
NOTA IMPORTANTE @ManyToMany:

Pregunta	Respuesta
¿Puedo poner @JoinTable en las dos clases?	❌ No
¿Da igual qué clase sea la propietaria?	✓ Sí, por funcionalidad
¿Importa cuál elijo?	✓ A nivel de diseño, pero no por Hibernate

¿Pero quién debería ser el propietario?
Hibernate **no obliga** a que sea una u otra. Pero lo recomendable es:
✓ **El que más sentido lógico tenga**
Ejemplos:

- Si un **Empleado** suele asignarse a proyectos, entonces **Empleado** es buen propietario.
- Si un **Proyecto** controla la asignación de empleados, entonces **Proyecto** sería el propietario.

Otro ejemplo de mapeo con anotaciones:



Ejemplo de mapeo anotaciones de @ManyToMany

❖ 4. Anotaciones adicionales de configuración de relaciones

- @JoinColumn:

Especifica la columna que se usará para la clave foránea en una relación de tipo uno a uno o muchos a uno.
Ejemplo:

```
@ManyToOne
@JoinColumn(name = "departamento_id", nullable = false)
private Departamento departamento;
```

- @JoinTable:

Define la tabla de unión y sus columnas en una relación muchos a muchos.
Especifica las columnas que almacenan las claves foráneas de ambas tablas.
Ejemplo:

```
@ManyToMany
@JoinTable(
    name = "empleado_proyecto",
    joinColumns = @JoinColumn(name = "empleado_id"),
    inverseJoinColumns = @JoinColumn(name = "proyecto_id")
)
private Set<Proyecto> proyectos;
```

Ejemplo completo de mapeo con anotaciones

A continuación se muestra un ejemplo de dos entidades (`Empleado` y `Departamento`) con sus relaciones mapeadas:

```
@Entity
@Table(name = "empleados")
public class Empleado {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nombre", nullable = false)
    private String nombre;

    @ManyToOne
    @JoinColumn(name = "departamento_id")
    private Departamento departamento;

    // Getters y setters
}

@Entity
@Table(name = "departamentos")
public class Departamento {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nombre", nullable = false)
    private String nombre;

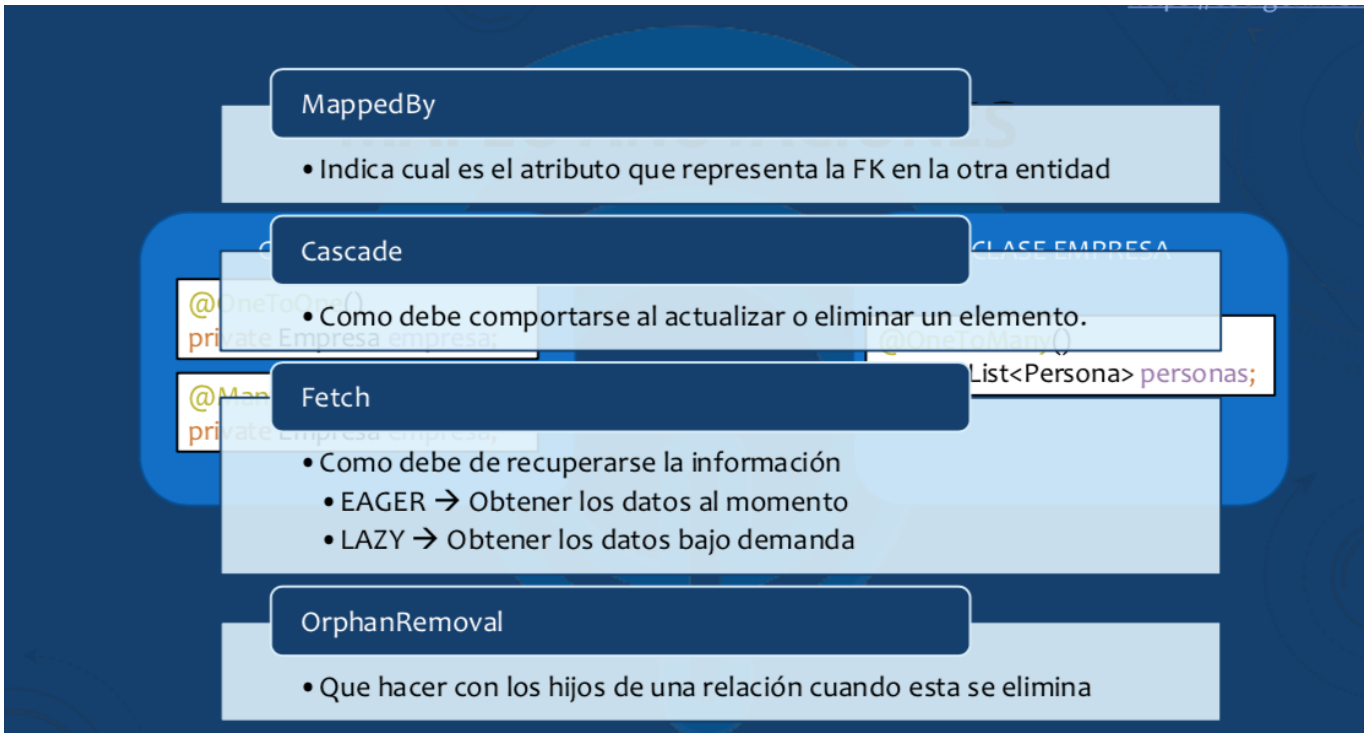
    @OneToMany(mappedBy = "departamento")
    private List<Empleado> empleados;

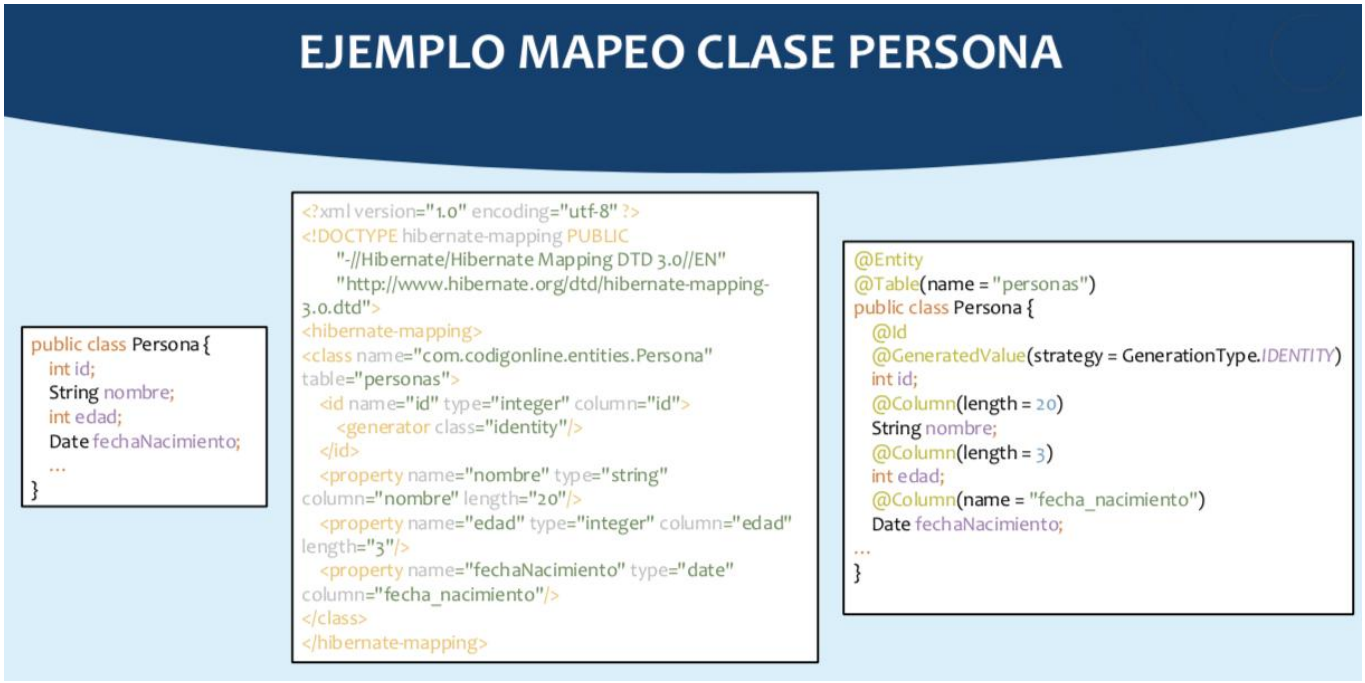
    // Getters y setters
}
```

En este ejemplo:

- `Empleado` tiene una relación muchos a uno con `Departamento` (varios empleados pueden pertenecer a un mismo departamento).
- `Departamento` tiene una relación uno a muchos con `Empleado` (un departamento puede tener varios empleados asociados).

Estas anotaciones simplifican y mejoran la legibilidad del código al permitir definir el esquema de la base de datos directamente en el modelo de datos en Java.





Esta imagen muestra un ejemplo de fichero de mapeo y de mapeo con anotaciones.

¿Qué es el patrón Singleton?

El **patrón Singleton** es un patrón de diseño creacional que garantiza que una clase tenga **solo una instancia** y proporciona un **punto de acceso global** a esa instancia. En aplicaciones de Hibernate, el Singleton es útil para **asegurar una única instancia de** SessionFactory, permitiendo reutilizar esta instancia en toda la aplicación y evitar así el costo de crear y destruir múltiples objetos de SessionFactory.

Implementación de Singleton en Hibernate

Antes de crear la clase main, una vez implementado hibernate, debemos crear una instancia que nos permita trabajar con la base de datos y que se utilizará a lo largo de toda la aplicación. A continuación, se muestra cómo implementar un Singleton para SessionFactory en Hibernate:

```
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class HibernateUtil {
    // Declaración de la instancia única de SessionFactory
    private static SessionFactory sessionFactory;
    // Bloque de inicialización estática
    static {
        try {
            // Inicialización de la única instancia de SessionFactory
            sessionFactory = new Configuration().configure().buildSessionFactory();
        } catch (Throwable ex) {
            System.err.println("Error al crear SessionFactory: " + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }
    // Método para obtener la instancia única de SessionFactory
    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
    // Método para cerrar la instancia de SessionFactory cuando ya no es necesaria
    public static void shutdown() {
        getSessionFactory().close();
    }
}
```

El patrón singletón se implementa creando en nuestra clase un método que crea una instancia del objeto solo si todavía no existe alguna. Para asegurar que la clase no puede ser instanciada nuevamente se regula el alcance del constructor (con atributos como protegido o privado). Es una clase que ayuda y accede a SessionFactory para obtener un objeto de sesión. La clase se llama configure() y carga sobre el fichero de configuración hibernate.cfg.xml y entonces construye la SessionFactory para obtener el objeto de sesión. El nombre de la clase es SessionFatoryUtil.java y se incluirá en la clase main.

Explicación de la implementación:

1. Variable sessionFactory:
   - Se declara como static y private, lo que garantiza que sea accesible solo dentro de la clase HibernateUtil y que exista una única instancia.
2. Inicialización en un bloque static:
   - El bloque static se ejecuta una sola vez cuando la clase es cargada por el ClassLoader. En este bloque, se crea la instancia de SessionFactory usando Configuration().configure().buildSessionFactory().
   - Se utiliza un try-catch para capturar y manejar cualquier excepción que pueda ocurrir durante la inicialización.
3. Método getSessionFactory():
   - Este método es el punto de acceso global para obtener la instancia de SessionFactory.
   - Siempre devuelve la misma instancia de SessionFactory creada en el bloque static.
4. Método shutdown():
   - Este método cierra la SessionFactory cuando ya no se necesita, liberando recursos. Es buena práctica llamarlo cuando la aplicación esté terminando o en operaciones de limpieza.

Uso del Singleton de SessionFactory:

Una vez definido el Singleton, se puede acceder a la SessionFactory desde cualquier parte de la aplicación sin necesidad de crear nuevas instancias. Por ejemplo:

```
import org.hibernate.Session;


public class MainApp {
    public static void main(String[] args) {
        // Obtener la sesión de Hibernate utilizando el Singleton
        Session session = HibernateUtil.getSessionFactory().openSession();


        // Realizar operaciones con la sesión
        session.beginTransaction();
        // ... operaciones CRUD
        session.getTransaction().commit();


        // Cerrar la sesión al finalizar
        session.close();


        // Cerrar la SessionFactory al terminar la aplicación
        HibernateUtil.shutdown();
    }
}
```

Mapeo de colecciones, relaciones y herencia

1. Mapeo de colecciones.

Hay bastantes rangos de mapeos que se pueden generar para colecciones que cubran diversos modelos relacionales.

El mapeo de colecciones en Hibernate permite asociar una entidad (clase Java) con una colección de datos (listas, conjuntos, mapas, etc.) relacionados en la base de datos. Estas colecciones suelen representarse como relaciones uno-a-muchos o muchos-a-muchos entre tablas.

Hibernate proporciona varias formas de mapear colecciones utilizando XML o anotaciones.

Tipos de Colecciones Soportadas:

- Listas (java.util.List): Se mapean con un orden específico y pueden usar índices.
- Conjuntos (java.util.Set): No permiten elementos duplicados.
- Mapas (java.util.Map): Almacenan pares clave-valor.
- Colas/Bag (java.util.Collection): Permiten duplicados, pero no tienen orden.

Tipos de Cascade para Colecciones. Hibernate soporta diferentes tipos de cascada para colecciones:

- ALL: Todas las operaciones (persistir, eliminar, refrescar, etc.) se aplican en cascada.
- PERSIST: Solo las operaciones de guardado.
- MERGE: Solo actualizaciones.
- REMOVE: Solo eliminaciones.

2. Mapeo de relaciones o asociaciones.

Para persistir, las relaciones usan las denominadas transacciones, ya que los cambios pueden incluir varias tablas. Una regla general para el mapeo es respetar el tipo de relación en el modelo de objetos y en el modelo relacional: así, una relación 1-1 en el modelo de objetos deberá corresponderse a una relación 1-1 en el modelo relacional.

Para mapear las relaciones, se usan los identificadores de objetos (OID). **Son la clave primaria** de la tabla relacionada **y** se agregan como una columna más en la tabla donde se quiere establecer la relación. Dicha columna es una **clave foránea** a la tabla con la que está relacionada.

3. Mapeo de herencia.

Como hemos comentado, las colecciones funcionan en ambos modelos, objeto y relacional. Para el caso de la herencia se presenta el problema que las bases de datos relacionales no la soportan. Así es que somos nosotros quienes debemos modelar como se verá la herencia en el modelo relacional. La decisión estará basada en el rendimiento y en la escalabilidad del modelo.

En Java podemos tener **herencia**:
class Persona { ... }
class Estudiante extends Persona { ... }
class Profesor extends Persona { ... }

Pero en una **base de datos relacional NO existe herencia**.
Las bases de datos solo conocen:

- Tablas, columnas, claves primarias, claves foráneas

Por eso, cuando usamos Hibernate, debemos definir **cómo queremos que se represente esa herencia en tablas**.
A eso se le llama: **Mapeo de Herencia (Inheritance Mapping)**
Hibernate ofrece **tres estrategias oficiales**:

1. modelar la jerarquía a una sola tabla (SINGLE TABLE).
2. modelar la jerarquía completa en tablas (JOINED TABLE) .
3. mapear cada tabla en tablas concretas (TABLE PER CLASS).

1. SINGLE TABLE (herencia en una sola tabla)

Todas las clases de la jerarquía se guardan en una única tabla.

Ejemplo (tabla única)

id	tipo	nombre	nota	salario
1	EST	Juan	8.5	null
2	PROF	Ana	null	2000

Ventajas:

- Muy rápido (solo 1 tabla)
- Consultas simples

Desventajas:

- Muchas columnas nulas
- Puede volverse una tabla “monstruo”

Anotación:

```
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "tipo")
```

2. JOINED TABLE (una tabla por clase, unidas por FK)

Cada clase tiene su **propia tabla**, y las subtipo están unidas con la tabla base mediante **clave foránea**.

Ejemplo de tablas:

Tabla persona

id	nombre
1	Juan
2	Ana

Tabla estudiante

id (FK persona.id)	nota
1	8.5

Tabla profesor

id (FK persona.id)	salario
2	2000

Ventajas:

- Diseño limpio
- No hay columnas nulas

Desventajas:

- Consultas más lentas (requiere JOINs)

Anotación:

@Inheritance(strategy = InheritanceType.JOINED)

3. TABLE PER CLASS (una tabla para cada clase hija)

Cada subclase tiene **su propia tabla completa**, con todas las columnas.

Ejemplo:

Tabla estudiante

id	nombre	nota
1	Juan	8.5

Tabla profesor

id	nombre	salario
2	Ana	2000

No existe una tabla "persona".

Ventajas:

- Tablas independientes y rápidas
- No hay columnas nulas

Desventajas:

- Duplicación de columnas
- Riesgo de inconsistencia si cambia un atributo común

Anotación:

@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)

Ejemplo: Mapeo Uno-a-Muchos (List)

Una entidad Departamento tiene una relación uno-a-muchos con una colección de Empleados.
Modelo Relacional:

- Tabla departamento
  - id_departamento (PK)
  - nombre
- Tabla empleado
  - id_empleado (PK)
  - nombre
  - id_departamento (FK)

1. Usando XML:

Mapeo en Departamento.hbm.xml:

```
<hibernate-mapping>
  <class name="com.miapp.Departamento" table="departamento">
    <id name="id" column="id_departamento">
      <generator class="native"/>
    </id>
    <property name="nombre" column="nombre"/>

    <!-- Mapeo de colección -->
    <list name="empleados" cascade="all" inverse="true">
      <key column="id_departamento" />
      <list-index column="indice" />
      <one-to-many class="com.miapp.Empleado" />
    </list>
  </class>
</hibernate-mapping>
```

Mapeo en Empleado.hbm.xml:

```
<hibernate-mapping>
  <class name="com.miapp.Empleado" table="empleado">
    <id name="id" column="id_empleado">
      <generator class="native"/>
    </id>
    <property name="nombre" column="nombre"/>

    <!-- Relación con Departamento -->
    <many-to-one name="departamento" class="com.miapp.Departamento" column="id_departamento" />
  </class>
</hibernate-mapping>
```

2. Usando Anotaciones (más moderno):

Entidad Departamento:

```
import javax.persistence.*;
import java.util.List;

@Entity
@Table(name = "departamento")
public class Departamento {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_departamento")
    private Long id;

    @Column(name = "nombre")
    private String nombre;

    // Relación uno-a-muchos con Empleado
    @OneToMany(mappedBy = "departamento", cascade = CascadeType.ALL)
    private List<Empleado> empleados;

    // Getters y setters
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public List<Empleado> getEmpleados() {
        return empleados;
    }
    public void setEmpleados(List<Empleado> empleados) {
        this.empleados = empleados;
```

```
    }
}
```

NOTA: Ejemplo con Cascade:

```
@OneToMany(mappedBy = "departamento", cascade = CascadeType.ALL)
private List<Empleado> empleados;
```

Esto asegura que, si eliminas un Departamento, todos los Empleado asociados también se eliminarán.

Entidad Empleado:

```
import javax.persistence.*;


@Entity
@Table(name = "empleado")
public class Empleado {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_empleado")
    private Long id;

    @Column(name = "nombre")
    private String nombre;

    // Relación muchos-a-uno con Departamento
    @ManyToOne
    @JoinColumn(name = "id_departamento")
    private Departamento departamento;

    // Getters y setters
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getNombre() {
        return nombre;
    }
```

```
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public Departamento getDepartamento() {
        return departamento;
    }
    public void setDepartamento(Departamento departamento) {
        this.departamento = departamento;
    }
}
```

Operación CRUD con Colecciones

Crear un Departamento con Empleados:

```
import org.hibernate.Session;
import org.hibernate.Transaction;
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        Session session = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = session.beginTransaction();

        // Crear departamento
        Departamento departamento = new Departamento();
        departamento.setNombre("TI");

        // Crear empleados
        Empleado emp1 = new Empleado();
        emp1.setNombre("Juan");
        emp1.setDepartamento(departamento);

        Empleado emp2 = new Empleado();
        emp2.setNombre("Ana");
        emp2.setDepartamento(departamento);

        // Añadir empleados a la colección
        List<Empleado> empleados = new ArrayList<>();
        empleados.add(emp1);
        empleados.add(emp2);
        departamento.setEmpleados(empleados);

        // Guardar en la base de datos
        session.save(departamento);

        tx.commit();
        session.close();
    }
}
```

Clases persistentes

Se denomina "persistencia" de los objetos a su capacidad para guardarse y recuperarse desde un medio de almacenamiento.

Una clase persistente es una clase cuyos objetos pueden:

- guardarse en un almacenamiento (normalmente una base de datos),
- recuperarse posteriormente,
- y mantener su estado incluso después de cerrar el programa.

En otras palabras: un objeto persistente no vive solo en la memoria RAM, sino que también se guarda en un sistema de almacenamiento.

Estados de un objeto persistente:

- No todas las instancias están siempre en la base de datos.
- Un objeto puede estar en varios estados:

Estado	Descripción
Transitorio	Aún no existe en la base de datos.
Persistente	Ya está guardado en la base de datos.
Separado (detached)	Fue persistente, pero ahora está desconectado del gestor de persistencia.

Las clases Java deberán tener las siguientes características:

- Deben tener un constructor público sin ningún tipo de argumentos.
- Para cada propiedad que queramos persistir debe haber un método get/set asociado.
- Implementar el interfaz serializable.

Una clase persistente:

- Equivale a una **tabla** de la base de datos
- Cada objeto de la clase es una **fila** dentro de esa tabla.

Un ejemplo podría ser el que se muestra a continuación:

```
package Persistente;
import java.util.HashSet;
import java.util.Set;
public class Departamentos implements java.io.Serializable {
private byte deptNo;

private String dnombre;
private String loc;
private Set empleadoses = new HashSet(0);
public Departamentos() {
}
public Departamentos(byte deptNo) {
   this.deptNo = deptNo;
}
public Departamentos(byte deptNo, String dnombre, String loc, Set
   empleadoses) {
   this.deptNo = deptNo;
   this.dnombre = dnombre;
   this.loc = loc;
   this.empleadoses = empleadoses;
}
   public byte getDeptNo() {
   return this.deptNo;
}
   public void setDeptNo(byte deptNo) {
   this.deptNo = deptNo;
}
   public String getDnombre() {
   return this.dnombre;
}
   public void setDnombre(String dnombre) {
   this.dnombre = dnombre;
}
   public String getLoc() {
   return this.loc;
}
   public void setLoc(String loc) {
```

```
    this.loc = loc;
}
    public Set getEmpleadoses() {
    return this.empleadoses;
}
    public void setEmpleadoses(Set empleadoses) {
    this.empleadoses = empleadoses;
}
    public void setdeptNo(byte b) {
    throw new UnsupportedOperationException("Not yet implemented");
}
```

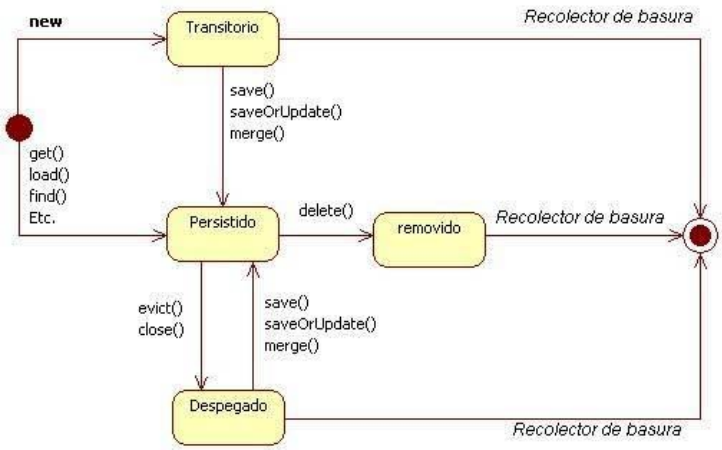
Sesiones; estados de un objeto

Para poder utilizar la persistencia en Hibernate es necesario definir un objeto Session utilizando la clase SesssionFactory. La sesión corresponde con un objeto que representa una unidad de trabajo con la base de datos. La sesión nos permite representar el gestor de persistencia, ya que dispone de una API básica que nos permite cargar y guardar objetos. La sesión está formada internamente por una cola de sentencias SQL que son necesarias ejecutar para poder sincronizar el estado de la sesión con la base de datos. Asimismo, la sesión contiene un lista de objetos persistentes. Un sesión corresponde con el primer nivel de caché.

Si para realizar el acceso a datos, usamos Hibernate, la sesión nos permite definir el alcance de un contexto determinado. Para poder utilizar los mecanismos de persistencia de Hibernate se debe inicializar el entorno Hibernate y obtener un objeto Session utilizando la clase SessionFactory de Hibernate. Un objeto Session Hibernate representa una única unidad-de-trabajo para un almacén de datos dado y lo abre un ejemplar de SessionFactory. Se deben cerrar las sesiones cuando se haya completado todo el trabajo de una transacción.

Los estados en los que se puede encontrar un objetos son:

- **Transitorio (Transient):** En este estado estará un objeto recién creado que no ha sido enlazado con el gestor de persistencia.
- **Persistente:** Ente este caso el objeto está enlazado con la sesión. Todos los cambios que se realicen será persistentes.
- **Disociado (Detached):** En este caso nos encontramos con un objeto persistente que sigue en memoria después de que termine la sesión. En este caso existe en Java y en la base de datos.
- **Borrado (Removed):** En esta caso el objeto está marcado para ser borrado de la base de datos. Existe en la aplicación Java y se borrará de la base de datos al terminar la sesión.



Un objeto en estado transitorio:

a) El objeto no ha sido enlazado con el gestor de persistencia.
b) El objeto ha sido marcado para ser borrado.
c) El objeto sigue en memoria después de finalizada la sesión.

Carga, almacenamiento y modificación de objetos

Carga

Para cargar un objeto de acceso a datos en la aplicación Java, el método load() de la clase Session suministra un mecanismo para capturar una instancia persistente, si conocemos su identificador. El método load() acepta un objeto Class, y cargará el estado de una nueva instancia de esa clase, inicializada en estado persistente.

El método load() lanzará una excepción irrecuperable si no existe la fila de base de datos correspondiente. Si no se está seguro de que exista una fila correspondiente, debe usarse el método get(), el cual consulta la base de datos inmediatamente y devuelve null si no existe una fila correspondiente

Existen dos métodos que se encargan de recuperar un objeto persistente por identificador:

- load() -> cuando sabes que el registro existe o solo necesitas una referencia para relaciones.
- get() -> cuando no estás seguro de que el registro exista. Seguro y directo.

La diferencia entre ellos radica en cómo indican que un objeto no se encuentra en la base de datos: get() devuelve un nulo y load() lanza una excepción ObjectNotFoundException.

Aparte de esta diferencia, load() intenta devolver un objeto proxy siempre y cuando le sea posible (no esté en el contexto de persistencia). Con lo que es posible que la excepción sea lanzada cuando se inicialice el objeto proxy. Esto es conocido como carga perezosa.

En el caso que se obtenga un proxy, no tiene impacto sobre la base de datos (no se ejecuta ninguna consulta), hasta que no se inicializa el mismo. Muy útil cuando se obtiene una referencia de un objeto para asociarlo a otro. (No es necesario obtener el objeto). Se modifica un objeto persistente.

En el siguiente enlace https://docs.jboss.org/hibernate/core/3.5/reference/es-ES/html/objectstate.html carga de objetos encontrarás todo lo referente a la carga, almacenamiento y modificación de objetos.

Almacenamiento y modificación de objetos

Para almacenar objetos persistentes se procesan siguiendo los siguientes pasos:

1. Se instancia un objeto nuevo (estado transitorio).
2. Se obtiene una sesión y se comienza la transacción, inicializando el contexto de persistencia.
3. Una vez obtenida la sesión, se llama al método save(), el cual introduce el objeto en el contexto de persistencia. Este método devuelve el identificador del objeto persistido.
4. Para que los cambios sean sincronizados en las bases de datos, es necesario realizar el commit de la transacción. Dentro del objeto sesión se llama al método flush(). Es posible llamarlo explícitamente. En este momento, se obtiene la conexión JDBC a la bases de datos para poder ejecutar la oportuna sentencia.
5. Finalmente, la sesión se cierra, con el objeto de liberar el contexto de persistencia, y por tanto, devolver la referencia del objeto creado al estado disociado.

Los objetos cargados, grabados, creados o consultados por las sesión pueden ser manipulados por la aplicación, y cualquier cambio a su estado de persistencia será persistido cuando se le aplique "flush" a la sesión. NOTA: Al hacer el commit() hace automáticamente un flush() antes de confirmar la transacción, por lo tanto no es necesario hacer el flush().

No hay que invocar ningún método en particular para que las modificaciones se vuelvan persistentes. Así que la manera más sencilla y directa de actualizar el estado de un objeto es cargarlo con load(), y luego manipularlo directamente, mientras la sesión esté abierta.

Para borrar objetos persistentes, podemos ejecutar Session.delete(), que quitará el estado de un objeto de la base de datos. Por supuesto, su aplicación podría aún contener una referencia al objeto quitado. Se puede borrar objetos en cualquier orden, no se van producir violaciones de llave externa, pero sí es posible violar constraints NOT NULL aplicadas a la columna de llave externa.

Muchas aplicaciones necesitan capturar un objeto en una transacción, mandarlo a la capa de interfaz de usuario para su manipulación, y grabar sus cambios en una nueva transacción. Las aplicaciones que usan este tipo de estrategia en entornos de alta concurrencia, normalmente usan datos versionados para garantizar aislamiento durante la "larga" unidad de trabajo.

Hibernate soporta este modelo, proveyendo "reasociación" de entidades desprendidas usando los métodos Session.update() o Session.merge().

Para sincronizar los datos con la base de datos es necesario realizar:

a) Un inicio de sesión.
b) Se invoca el método save().
c) Se invoca al método flush().

- **REGLA DE ORO de Hibernate**
  Siempre que vayas a hacer una operación (insert, update o delete):
  1. Utiliza session.beginTransaction();
  2. Llama a **save()**, **update()** o **delete()**.
  3. Realiza el session.getTransaction().commit();

  Hibernate ya hace flush() por ti en el commit.
  No debes escribirlo tú.

- Ejemplo Inserción con Hibernate:

```
// Abrimos transacción
session.beginTransaction();

//Creamos la persona
Persona persona = new Persona("Alvaro", 27, Persona.toDate("25/11/1993"));

//Guardamos
  session.save(persona);

// Cerramos transacción
session.getTransaction().commit();
```

Una vez realizado todo el mapeo y configuración inicial realizar inserciones es muy sencillo.
Para guardar deberemos utilizar el método save del objeto session, este se encargará de guardar toda la información necesaria en la BD. Un punto muy importante es que todas las operaciones deben ir en transacciones por lo que tendremos los 3 pasos mencionados anteriormente.

- Ejemplo Modificación con Hibernate:

```
// Abrimos transacción
session.beginTransaction();

// Obtenemos la persona por su id (por ejemplo, id = 1)
Persona persona = session.get(Persona.class, 1);

// Modificamos un campo
if(persona != null) {
   persona.setEdad(28);
   session.update(persona);  // Persistimos el cambio
}

// Cerramos transacción
session.getTransaction().commit();
```

Para realizar las actualizaciones disponemos de dos mecanismos diferentes:
- Utilizar la función update (Será el mecanismo utilizado de forma "normal")
- Utilizar la función save (Esta función en el update está reservada a cuando hemos recuperado un objeto previamente de la BD, ya sea porque hemos hecho un select o una inserción nueva). Si nosotros le ponemos el ID de forma manual no funcionará

✓ Si recuperamos y actualizamos podremos utilizar el save.

- Ejemplo Eliminación con Hibernate:

```
// Abrimos transacción
session.beginTransaction();

// Obtenemos la persona que queremos eliminar
Persona persona = session.get(Persona.class, 1);

if(persona != null) {
   session.delete(persona); // Eliminamos del registro
}

// Cerramos transacción
session.getTransaction().commit();
```

Para realizar las eliminaciones deberemos recurrir al método delete.

Consultas SQL

Hemos visto que las inserciones y eliminaciones con hibernate son increíblemente sencillas, solo debemos pasarle el objeto a eliminar. ¿Pero que pasa con las búsquedas?

Usando Hibernate, la ejecución de consultas SQL nativas se controla por medio de la interfaz SQLQuery, la cual se obtiene llamando a Session.createSQLQuery(). Las siguientes secciones describen cómo utilizar esta API para consultas.

La consulta SQL más básica es para obtener a una lista de escalares (valores).

- sess.createSQLQuery("SELECT * FROM Personas").list();
- sess.createSQLQuery("SELECT id, nombre, edad FROM Personas").list();

Estas retornarán una lista de objetos arrays (Object[]) con valores escalares para cada columna en la tabla PERSONAS. Hibernate utilizará ResultSetMetadata para deducir el orden real y los tipos de los valores escalares retornados.

Otro tipo de consulta más compleja, es la consulta de entidades. Para obtener los objetos entidades desde una consulta sql nativa, se utiliza por medio de addEntity().

- sess.createSQLQuery("SELECT * FROM PERSONAS").addEntity(Persona.class);
- sess.createSQLQuery("SELECT ID,NOMBRE,EDAD FROM PERSONAS").addEntity(Persona.class);

Se especifica esta consulta:

- La cadena de consulta SQL.
- La entidad devuelta por la consulta.

Asumiendo que Persona es mapeado como una clase con las columnas IDENTIFICACION, NOMBRE y FECHA DE NACIMIENTO las consultas anteriores devolverán una Lista en donde cada elemento es una entidad Persona.

¿En qué estado se encuentra un objeto persistente en memoria, una vez finalizada la sesión?

a) Transitorio.
b) Borrado.
c) Disociado.

Lenguajes propios de la herramienta ORM: HQL

Para la realización de consultas "select" deberemos hacer uso del lenguaje propio de Hibernate: HQL (Hibernate Query Language).

Hibernate utiliza este lenguaje de consulta potente (HQL) que se parece a SQL. Sin embargo, comparado con SQL, HQL es completamente orientado a objetos y comprende nociones como herencia, polimorfismo y asociación. Las consultas se escriben en HQL y Hibernate se encarga de convertirlas al SQL usado por la base de datos con la que estemos trabajando y ejecutarla para realizar la operación indicada.

HQL es case-insensitive, o sea que sus sentencias pueden escribirse en mayúsculas y minúsculas. Por lo tanto "SeLeCt", "seleCT", "select", y "SELECT" se entienden como la misma cosa. Lo único con lo que debemos tener cuidado es con los nombres de las clases que estamos recuperando y con sus propiedades, ahí si se distinguen mayúsculas y minúsculas. O sea, en este caso "pruebas.Hibernate.Usuario" NO ES LO MISMO que "PrueBAs.Hibernate.UsuArio".

Entre las características más importantes de HQL.

- Soporte completo para operaciones relacionales: HQL permite representar consultas SQL en forma de objetos. HQL usa clases y atributos o propiedades en vez de tablas y columnas.
- Regresa sus resultados en forma de objetos: Las consultas realizadas usando HQL regresan los resultados de las mismas en la forma de objetos o listas de objetos, que son más fáciles de usar.
- Consultas Polimórficas: Podemos declarar el resultado usando el tipo de la superclase e Hibernate se encargara de crear los objetos adecuados de las subclases correctas de forma automática.
- Soporte para características avanzadas: HQL contiene muchas características avanzadas que son muy útiles y que no siempre están presentes en todas las bases de datos, o no es fácil usarlas, como paginación, fetch joins con perfiles dinámicos, inner y outer joins, etc. Además soporta proyecciones, funciones de agregación (max, avg), y agrupamientos, ordenamientos, y subconsultas.
- Independiente del manejador de base de datos: Las consultas escritas en HQL son independientes de la base de datos (siempre que la base de datos soporte la característica que estamos intentando utilizar)

El lenguaje de consulta propio de Hibernate es:

a) HQL.
b) SQL.
c) Java.

Sintaxis del lenguaje HQL

El Hibernate Query Languaje (HQL) es el lenguaje de consultas que usa Hibernate para obtener los objetos desde la base de datos. Su principal particularidad es que las consultas se realizan sobre los objetos java que forman nuestro modelo de negocio, es decir, las entidades que se persisten en Hibernate. Ésto hace que HQL tenga las siguientes características:

- Los tipos de datos son los de Java.
- Las consultas son independientes del lenguaje de SQL específico de la base de datos
- Las consultas son independientes del modelo de tablas de la base de datos.
- Es posible tratar con las colecciones de Java.
- Es posible navegar entre los distintos objetos en la propia consulta.

En Hibernate las consultas HQL se lanzan (o se ejecutan) sobre el modelo de entidades que hemos definido en Hibernate, esto es, sobre nuestras clases de negocio.
Las sentencias que se estudian en este capitulo son del lenguaje HQL de Hibernate, en gran parte son compatibles con JPQL (Java Pesistance Query Language), lenguaje de consulta de persistencia de Java.

De forma poco ortodoxa se podría ver cómo que nuestro modelo de tablas en HQL son las clases Java y NO las tablas de la base de datos. Es decir que cuando hagamos ”SELECT columna FROM nombreTabla” , el ”nombreTabla” será una clase Java y ”columna” será una propiedad Java de dicha clase y nunca una tabla de la base de datos ni una columna de una tabla.

Sentencias consulta (SELECT)

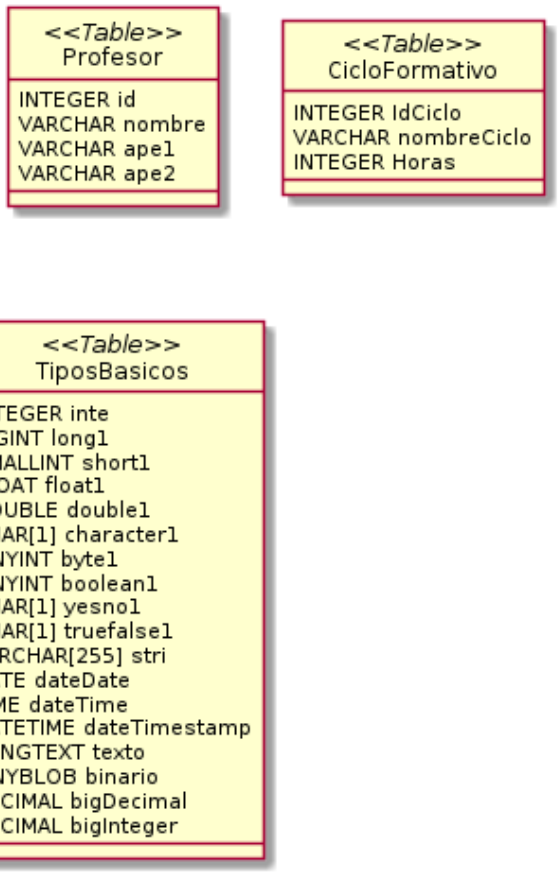
- **Clausula from:** La consulta más simple que se puede realizar con Hibernate, es utilizando la cláusula from, la siguientes sería una consulta que mostraría todos los datos de una tabla de nombre Alumnos: from Alumnos
- **Cláusula select:** La cláusula select escoge qué objetos y propiedades devolver en el conjunto de resultados de la consulta. Un ejemplo de consulta podría ser select alumno.nombre from Alumnos alumno where alumno.nombre like 'A%'
- **La cláusula where:** La cláusula where nos permite refinar la lista de instancias retornadas. Si no existe ningún alias, puede referirse a las propiedades por nombre: from Alumnos where nombre='Francisco'. Si existe un alias, usaremos un nombre de propiedad calificado: from Alumnos as alumnos where alumnos.nombre='Francisco'. Esto retorna instancias de Alumnos llamados "Francisco".
- **Funciones de agregación:** Las consultas HQL pueden retornar resultados de funciones de agregación sobre propiedades: select avg(alumnos.nota), sum(alumnos.nota), max(alumnos.nota), count(alumnos) from Alumnos alumnos.
- **Expresiones:** Las expresiones utilizadas en la cláusula where incluyen lo siguiente: operadores matemáticos, operadores de comparación binarios, operadores lógicos , paréntesis ( ) que indican agrupación, funciones Java, etc.
- **La cláusula order by:** La lista retornada por una consulta se puede ordenar por cualquier propiedad de una clase retornada o componentes. La palabra asc o desc opcionales indican ordenamiento ascendente o descendente respectivamente.

- **La cláusula group by:** Una consulta que retorna valores agregados se puede agrupar por cualquier propiedad de una clase retornada o componentes.
- **La cláusula having:** restringe grupos.

La sintaxis HQL para las sentencias de consultas es la siguiente:

```
[select_clause] from_clause [where_clause] [groupby_clause] [having_clause] [orderby_clause]
```

Sobre el siguiente modelo de datos que se expone a continuación se harán todos los ejemplos de HQL



Vemos en la sintaxis que la única clausula obligatoria es from_clause.
No obstante, Hibernate recomienda incluir select_clause , por portabilidad con JPQL (java persistence query languagge).

Veamos ahora un sencillo ejemplo de consulta en HQL:

```
SELECT c FROM Ciclo c ORDER BY nombre
```

¿Qué diferencias podemos ver entre HQL y SQL?

- Ciclo hace referencia a la clase Javaejemplo02.Ciclo y NO a la tabla CicloFormativo. Nótese que la clase Java y la tabla tienen distinto nombre .
- Es necesario definir el alias c de la clase Java Ciclo.
- Tras la palabra SELECT se usa el alias en vez del ”*”.
- Al ordenar los objetos se usa la propiedad nombre de la clase Ciclo en vez de la columna nombreCiclo de la tabla CicloFormativo.

Recuerda incluir el alias en la consulta HQL. Si no se hace y se deja la consulta de la siguiente forma:

```
SELECT Ciclo FROM Ciclo
```

se producirá la siguiente excepción:

```
java.lang.NullPointerException
```

Forma correcta:

```
SELECT c FROM Ciclo c
```

Hibernate soporta no incluir la parte del SELECT en la consulta HQL, quedando entonces la consulta de la siguiente forma:

```
FROM Ciclo
```

pero en la propia documentación se recomienda no hacerlo ya que de esa forma se mejora la portabilidad en caso de usar el lenguaje de consultas de JPA .

Se ha hecho mención de esta característica ya que en muchos tutoriales que se encuentran por Internet se hace uso de ella.

Mayúsculas

Respecto a la sensibilidad de las mayúsculas y minúsculas , el lenguaje HQL sí que lo es, pero con matices.

- Las palabras clave del lenguaje NO son sensibles a las mayúsculas o minúsculas.
    - Las siguientes 2 consultas son equivalentes.

```
select count(*) from Ciclo
SELECT COUNT(*) FROM Ciclo
```

- El nombre de las clases Java y sus propiedades SI son sensibles a las mayúsculas o minúsculas.
    - La siguiente consulta HQL es correcta

```
SELECT c.nombre FROM Ciclo c WHERE nombre='Desarrollo de aplicaciones Web'
```

    - La siguiente consulta HQL es **errónea** ya que la propiedad nombre está escrita con la "N" en mayúsculas.

```
SELECT c.Nombre FROM Ciclo c WHERE Nombre='Desarrollo de aplicaciones Web'
```

    - La siguiente consulta HQL es errónea ya que el nombre de la clase Java Ciclo está escrita con la "c" en minúsculas.

```
SELECT c.nombre FROM ciclo c WHERE nombre='Desarrollo de aplicaciones Web'
```

- Al realizar comparaciones con los valores de las propiedades, éstas NO son sensibles a las mayúsculas o minúsculas.
    - Las siguientes 2 consultas retornan los mismos objetos

```
SELECT c.nombre FROM Ciclo c WHERE nombre='Desarrollo de aplicaciones Web'
SELECT c.nombre FROM Ciclo c WHERE nombre='DESARROLLO DE APLICACIONES WEB'
```

Filtrado

Al igual que en SQL en HQL también podemos filtrar los resultados mediante la cláusula WHERE. La forma de usarla es muy parecida a SQL.

```
SELECT p FROM Profesor p WHERE nombre='ISABEL' AND ape1<>'ORELLAN
```

Al igual que con el nombre de la clase, el nombre de los campos del WHERE siempre hace referencia a las propiedades Java y nunca a los nombres de las columnas. De esa forma seguimos independizando nuestro código Java de la estructura de la base de datos.

Literales

- Texto

El carácter para indicar un literal de texto es la comilla simple no pudiéndose usar la doble comilla.

```
SELECT p FROM Profesor p WHERE nombre='juan'
```

Si se quiere usar la comilla dentro de un literal deberemos duplicarla.

```
SELECT p FROM Profesor p WHERE ape1='perez l''andreu'
```

- Integer

Para incluir un número del tipo integer simplemente se escribe dicho número.

```
SELECT tb FROM TiposBasicos tb WHERE inte=4
```

- Long

Para incluir un número del tipo long se escribe dicho número y se añade una L mayúscula al final.

```
SELECT tb FROM TiposBasicos tb WHERE long1=4L
```

- double

Para representar un double se escribe el número separando la parte decimal con un punto o se puede usar la notación científica.

```
SELECT tb FROM TiposBasicos tb WHERE double1=1.45
SELECT tb FROM TiposBasicos tb WHERE double1=1.7976931348623157E308
```

- float

Para representar un float se escribe el número separarando la parte decimal con un punto o se puede usar la notación científica pero se le añade el carácter F en mayúscula al final.

```
SELECT tb FROM TiposBasicos tb WHERE float1=1.45F
SELECT tb FROM TiposBasicos tb WHERE float1=3.4028235E38F
```

- Fecha

Para indicar una fecha la incluiremos entre comillas simples con el formato yyyy-mm-dd

```
SELECT tb FROM TiposBasicos tb WHERE dateDate='2012-07-25'
```

- Hora

Para indicar una hora la incluiremos entre comillas simples con el formato hh:mm:ss

```
SELECT tb FROM TiposBasicos tb WHERE dateTime='02:05:10'
```

- fecha y hora

Para indicar una fecha y hora la incluiremos entre comillas simples con el formato yyyy-mm-dd hh:mm:ss.millis,siendo optativos el último punto y los milisegundos .

```
SELECT tb FROM TiposBasicos tb WHERE dateTime='2012-07-25 02:05:10'
```

- Operadores de comparación

Para comparar los datos en una expresión se pueden usar las siguientes Operadores:

- ✓ Signo igual ”=”: La expresión será verdadera si los dos datos son iguales. En caso de comparar texto, la comparación no es sensible a mayúsculas o minúsculas.
- ✓ Signo mayor que ”>”: La expresión será verdadera si el dato de la izquierda es mayor que el de la derecha.
- ✓ Signo mayor que ”>=”: La expresión será verdadera si el dato de la izquierda es mayor o igual que el de la derecha.
- ✓ Signo mayor que ”<”: La expresión será verdadera si el dato de la izquierda es menor que el de la derecha.
- ✓ Signo mayor que ”<=”: La expresión será verdadera si el dato de la izquierda es menor o igual que el de la derecha.
- ✓ Signo desigual ”<>”: La expresión será verdadera si el dato de la izquierda es distinto al de la derecha.
- ✓ Signo desigual ”!=”: La expresión será verdadera si el dato de la izquierda es distinto al de la derecha.
- ✓ Operador ”between”: La expresión será verdadera si el dato de la izquierda está dentro del rango de la derecha.

```
SELECT tb FROM TiposBasicos tb WHERE inte BETWEEN 1 AND 10
```

- ✓ Operador ”in”: La expresión será verdadera si el dato de la izquierda está dentro de la lista de valores de la derecha.

```
SELECT tb FROM TiposBasicos tb WHERE inte IN (1,3,5,7)
```

- ✓ Operador ”like”: La expresión será verdadera si el dato de la izquierda coincide con el patrón de la derecha.Se utilizan los mismos signos que en SQL ”%” y ”_”.

```
SELECT tb FROM TiposBasicos tb WHERE stri LIKE 'H_la%'
```

- ✓ Operador ”not”: Niega el resultado de una expresión.
- ✓ Expresión ”is null”: Comprueba si el dato de la izquierda es null.

```
SELECT tb FROM TiposBasicos tb WHERE dataDate IS NUL
```

- Operadores lógicos

Se puede hacer uso de los típicos operadores lógicos como en SQL: AND, OR y NOT

```
SELECT p FROM Profesor p WHERE nombre='ANTONIO' AND (ape1='LARA' OR ape2='RUBIO')
```

- Operadores Aritméticos

Se puede hacer uso de los típicos operadores aritméticos: suma (+), resta (-), multiplicación (*) y división (/).

```
SELECT tb FROM TiposBasicos tb WHERE (((inte+1)*4)-10)/2=1
```

- Funciones de agregación

Las funciones de agregación que soporta HQL son:

- ✓ AVG(): Calcula el valor medio de todos los datos.
- ✓ SUM(): Calcula la suma de todos los datos.
- ✓ MIN(): Calcula el valor mínimo de todos los datos.
- ✓ MAX(): Calcula el valor máximo de todos los datos.
- ✓ COUNT(): Cuanta el número de datos.

```
SELECT AVG(c.horas),SUM(c.horas),MIN(c.horas),MAX(c.horas),COUNT(*) FROM Ciclo c
```

- Funciones sobre escalares

Algunas de las funciones que soporta HQL sobre datos escalares son:

- ✓ UPPER(s): Transforma un texto a mayúsculas.
- ✓ LOWER(s): Transforma un texto a minúsculas.
- ✓ CONCAT(s1, s2): Concatena dos textos
- ✓ TRIM(s):Elimina los espacio iniciales y finales de un texto.
- ✓ SUBSTRING(s, offset, length): Retorna un substring de un texto.El offset empieza a contar desde 1 y no desde 0.
- ✓ LENGTH(s): Calcula la longitud de un texto.
- ✓ ABS: Calcula el valor absoluto de un número.
- ✓ SQRT: Calcula la raíz cuadrada del número
- ✓ Operador ”||” : Permite concatenar texto.

```
SELECT p.nombre || ' ' || p.ape1 || ' ' || p.ape2 FROM Profesor p WHERE Id=1001
```

- Ordenación

Como en SQL también es posible ordenar los resultados usando ORDER BY. Su funcionamiento es como en SQL.

```
SELECT p FROM Profesor p ORDER BY nombre ASC,ape1 DESC
```

Las palabras ASC y DESC son opcionales al igual que en SQL.

El uso de funciones escalares y funciones de agrupamiento en la cláusula ORDER BY sólo es soportado por Hibernate si es soportado por el lenguaje de SQL de la base de datos sobre la que se está ejecutando.
No se permite el uso de expresiones aritméticas en la cláusula ORDER BY.

- Agrupaciones

Al igual que en SQL se pueden realizar agrupaciones mediante las palabras claves GROUP BY y HAVING

```
SELECT nombre,count(nombre) FROM Profesor p GROUP BY nombre HAVING count(nombre)>1 ORDER BY count(nombre)
```

Los nombres de profesores que se repiten mas de una vez

El uso de funciones escalares y funciones de agrupamiento en la cláusula HAVING sólo es soportado por Hibernate si es soportado por el lenguaje de SQL de la base de datos sobre la que se está ejecutando.
No se permite el uso de expresiones aritméticas en la cláusula GROUP BY.

- Subconsultas

HQL también soporta subconsultas como en SQL.

```
SELECT c.nombre,c.horas FROM Ciclo c WHERE c.horas > (SELECT AVG(c2.horas) FROM Ciclo c2)
```

Ciclos que duran más horas que la media de duración de todos los ciclos

- Combinaciones (JOIN)

En la clausula from podemos incluir múltiples clases, lo que causa un producto cartesiano o una unión "cruzada" (cross join).

```
FROM Profesor as profesor, Ciclo as ciclo
```

Para poder relacionar los profesores con los ciclos que imparten tenemos la opción de especificar la condición de combinación, de forma similar a como se hace en SQL

```
FROM Profesores as profesor, Ciclo as ciclo
WHERE profesor = ciclo.profesor
```

También podemos utilizar la clausula JOIN

```
FROM Profesor as profesor INNER JOIN profesor.ciclo as  ciclo
```

En HQL (Hibernate Query Language), la operación JOIN se utiliza para realizar consultas que combinan datos de dos o más entidades relacionadas, aprovechando las asociaciones mapeadas en Hibernate. Esto permite trabajar con objetos en lugar de tablas, simplificando el acceso a datos relacionales.

Aquí tienes una descripción básica y ejemplos de cómo usar JOIN en HQL:

- **Consulta básica con JOIN**

Supongamos que quieres listar estudiantes junto con sus escuelas:

```
FROM Student s JOIN s.school sc
```

- **Consulta con LEFT JOIN**

Si también deseas incluir estudiantes que no están asociados a ninguna escuela:

```
FROM Student s LEFT JOIN s.school sc
```

En HQL (Hibernate Query Language), la cláusula ON no es necesaria ni comúnmente utilizada como lo sería en SQL estándar, porque las asociaciones entre entidades están implícitas en los mapeos definidos en las clases de entidad de Hibernate.

- **Consulta JOIN con una condición adicional (simulando un ON)**

En lugar de utilizar ON, HQL utiliza las relaciones mapeadas en las entidades para realizar los JOIN. Sin embargo, si deseas agregar condiciones específicas a un JOIN, puedes colocarlas en la cláusula WHERE o integrarlas dentro de la propia declaración de JOIN. Esto es conceptualmente similar al uso de ON en SQL.

```
FROM Employee e JOIN e.department d WHERE d.name = 'IT'
```

Los tipos de uniones soportadas se tomaron prestados de ANSI SQL

- ✓ inner join
- ✓ left outer join
- ✓ right outer join
- ✓ full join.

- Parámetros nombrados

En ejemplos anteriores incluíamos directamente el valor de los parámetros del WHERE. Eso nos llevaría desde Java a tener un código similar al siguiente:

```
String nombre="ISIDRO";
String ape1="CORTINA";
String ape2="GARCIA";
Query query = session.createQuery("SELECT p FROM Profesor p where nombre='" + nombre +"' AND  ape1='" + ape1 + "' AND  ape2='" + ape2 + "' ");
List<Profesor> profesores = query.list();
for (Profesor profesor : profesores) {
    System.out.println(profesor.toString());
}
```

Hibernate soporta parámetros nombrados en sus consultas HQL. Esto facilita la escritura de consultas HQL que aceptan la entrada del usuario. A continuación se muestra la sintaxis simple de usar parámetros nombrados:

```
String nombre="ISIDRO";
String ape1="CORTINA";
String ape2="GARCIA";
 Query query = session.createQuery("SELECT p FROM Profesor p where nombre=:nombre +"' AND  ape1=:ape1 + "' AND  ape2=:ape2 ");
 List<Profesor> profesores = query.setParameter(1,nombre).setParameter(2,ape1).setParameter(3,ape2).list();
 for (Profesor profesor : profesores) {
     System.out.println(profesor.toString());
 }
```

La asignación de valores a los parámetros se realiza con el método setParameter, los métodos setString, setByte, etc. están obsoletos. También podemos utilizar parámetros con ?.

En la siguiente dirección Web, encontrarás una amplia información sobre el lenguaje de consulta HQL.
https://docs.jboss.org/hibernate/orm/3.5/reference/es-ES/html/queryhql.html

Consultas por criterios

Hibernate tiene el objeto Query que nos da acceso a todas las funcionalidades para poder leer objetos desde la base de datos. Veamos ahora un sencillo ejemplo de cómo funciona y posteriormente explicaremos más funcionalidades de la clase Query.

Sintaxis básica para recuperar todos los datos con HQL:

```
String hql = "FROM Profesor";
Query query = session.createQuery(hql);
List<Empleado> empleados = query.getResultList();
```

Aunque no es necesario especificar SELECT en HQL, también se puede hacer para mayor claridad, así:

```
//Crear consulta HQL
Query query = session.createQuery("SELECT p FROM Profesor p");
//Ejecutar la consulta y obtener los resultados
Lis<Profesor> profesores = query.getResultList();
//Iterar sobre los resultados
for (Profesor profesor : profesores)
{   System.out.println(profesor.toString());
}
```

Lanzar una consulta con Hibernate es bastante simple. Usando la session llamamos al método createQuery(String queryString) con la consulta en formato HQL y nos retorna un objeto. Después, sobre el objeto Query llamamos al método getResultList () que nos retorna una lista de los objetos que ha retornado.

Por último podemos ver cómo usar la lista de objetos Profesor aunque este código ya es simplemente el uso de la clase java.util.List que no tiene nada que ver con Hibernate.

Recuperar un único resultado por ID o alguna otra condición:

Supongamos que queremos recuperar un Empleado específico por su id.

```
String hql = "FROM Empleado e WHERE e.id = :id";
Query query = session.createQuery(hql);
query.setParameter("id", empleadoId);  // Asumiendo que "empleadoId" contiene el ID del empleado

Empleado empleado = (Empleado) query.getSingleResult();
```

Recuperar un valor específico (consulta de proyección):

Si quieres recuperar un único valor o un campo específico, puedes utilizar una proyección en HQL. Supongamos que queremos recuperar el nombre de un Empleado por su id.

```
String hql = "SELECT e.nombre FROM Empleado e WHERE e.id = :id";

Query query = session.createQuery(hql);

query.setParameter("id", empleadoId);


String nombre = (String) query.getSingleResult();
```

1. Query

A partir de Hibernate 5, el uso de org.hibernate.Query ha sido desalentado en favor de la implementación de la interfaz javax.persistence.Query. Esto se debe a la adopción de JPA como estándar.

Por lo tanto, la interfaz org.hibernate.Query no es estrictamente obsoleta, pero se recomienda usar la interfaz de JPA (javax.persistence.Query) para las consultas, ya que es más estándar y compatible con otros proveedores de JPA.

Resumen de funcionalidades clave de javax.persistence.Query:

- Crear consultas: createQuery (JPQL) y createNativeQuery (SQL nativo).
- Establecer parámetros: setParameter para pasar valores dinámicos.
- Obtener resultados: getResultList() (para múltiples resultados) y getSingleResult() (para un solo resultado).
- Paginación: setMaxResults(int) y setFirstResult(int) para controlar los resultados devueltos.
- Consultas de actualización y eliminación: executeUpdate() para modificar datos.
- Consultas de agregación: Funciones como COUNT(), AVG(), SUM(), etc.
- Manejo de excepciones: Capturar excepciones como NoResultException, NonUniqueResultException.

2. Consultas con nombre

En cualquier libro sobre arquitectura del software siempre se indica que las consultas a la base de datos no deberían escribirse directamente en el código sino que deberían estar en un fichero externo para que puedan modificarse fácilmente.

Hibernate provee una funcionalidad para hacer ésto mismo de una forma sencilla. En cualquier fichero de mapeo de Hibernate se puede incluir anotaciones @NamedQuery and @NamedQueries Annotations con las consultas HQL que deseamos lanzar.

En el siguiente ejemplo se ha definido una query nombrada para la clase Profesor:

```
@NamedQuery(name="Profesores.findAll", query="SELECT p FROM Profesor p")
```

La anotación @NamedQuery contiene cuatro parámetros, dos de los cuales son obligatorios y dos opcionales. Los dos parámetros requeridos, name y query definen el nombre de la consulta y la propia cadena de consulta y se muestran arriba. Los dos elementos opcionales, lockMode y hints, proporcionan un reemplazo estático para los métodos setLockMode y setHint.

La entidad Profesor quedaría:

```
@Entity
@NamedQuery(name="Profesores.findAll", query="SELECT p FROM Profesor p")
public class Profesor {
  ...
}
```

Para añadir varias consultas nombradas , tiene que haber una consulta nombrada por cada una de ellas y estas dentro de una anotación envoltorio @NamedQueries

```
@Entity
@NamedQueries({
@NamedQuery(name="Profesores.findAll", query="SELECT p FROM Profesor p")
@NamedQuery(name="Profesores.ByName", query="SELECT p FROM Profesor p where p.nombre =:nombre and p.ape1=:ape1 and p.ape2=:ape2")
})
public class Profesor {
  ...
}
```

Estas anotaciones tienen los siguientes datos:

- name: Este atributo define el nombre de la consulta.Es el nombre que posteriormente usaremos desde el código Java para acceder a la consulta.
- query es la consulta en formato HQL que ejecutará Hibernate.
- :nombre, :ape1, :ape2 son parámetros nombrados, también se podrían especificar con parámetros posicionales ? como en JDBC:

```
"SELECT p FROM Profesor p where p.nombre =? and p.ape1=? and p.ape2=?"
```

3. Código Java

Para hacer uso de una consulta con nombre usaremos el método getNamedQuery(String queryString) en vez de createQuery(String queryString) para obtener el objeto Query. Por lo tanto sólo se ha modificado la línea 1 y el resto del código queda exactamente igual.

```
Query query = session.getNamedQuery("Profesores.findAll");
List<Profesor> profesores = query.list();
for (Profesor profesor : profesores) {
System.out.println(profesor.toString());
}
```

Como podemos ver el uso de consultas con nombre es muy sencillo al usar Hibernate.

```
String nombre="ISIDRO";
String ape1="CORTINA";
String ape2="GARCIA";
Query query = session.getNamedQuery(Profesores.ByName)
query.setString("nombre",nombre);
query.setString("ape1",ape1);
query.setString("ape2",ape2);
List<Profesor> profesores = query.list();
for (Profesor profesor : profesores) {
                System.out.println(profesor.toString());
}
```

Gestión de Transacciones

Un transacción es un conjunto de órdenes que se ejecutan formando un unidad de trabajo, en forma indivisible o atómica.

Para la gestión de transacciones en Hibernate, no se produce bloqueo de objetos en la memoria. La aplicación puede esperar el comportamiento definido por el nivel de aislamiento de sus transacciones de las bases de datos. Gracias a la Session, la cual también es un caché con alcance de transacción. Para realizar con éxito la gestión de transacciones, ésta se van a basar en el uso del objeto Session.

El objeto Session se obtiene a partir de un objeto SessionFactory ,invocando el método openSession. Un objeto SessionFactory representa una configuración particular de un conjunto de metadatos de mapping objeto/relaciona.

```
// Abrimos la sesión con try-with-resources
try (Session session = HibernateUtil.getSessionFactory().openSession()) {

    // Iniciamos la transacción
    Transaction tx = session.beginTransaction();

    // ================================================
    // 1) INSERT
    // ================================================
    Persona nueva = new Persona("Alvaro", 27, Persona.toDate("25/11/1993"));
    session.save(nueva);
    System.out.println("Insertado ID: " + nueva.getId());

    // ================================================
    // 2) READ
    // ================================================
    Persona persona = session.get(Persona.class, nueva.getId());
    if (persona != null) {
        System.out.println("Recuperado: " + persona.getNombre());
    } else {
        System.out.println("No existe la persona");
    }
```

```
    // ================================================
    // 3) UPDATE
    // ================================================
    persona.setEdad(30);
    session.update(persona);
    System.out.println("Persona actualizada");

    // ================================================
    // 4) DELETE
    // ================================================
    session.delete(persona);
    System.out.println("Persona eliminada");

    // Confirmamos la transacción
    tx.commit();

} catch (Exception e) {
    e.printStackTrace();
    // Si ocurre un error, Hibernate automáticamente cierra la sesión
    // La transacción NO se puede revertir fuera del try,
    // así que es buena práctica crear la transacción dentro del try
}

// Al finalizar la aplicación
HibernateUtil.shutdown();
```

Cuando se crea el objeto Session , se le asigna la conexión de la base de datos que va a utilizar. Una vez obtenido el objeto Session , se crea una nueva unidad de trabajo (Transaction) utilizando el método beginTransaction. Dentro del contexto de la transacción creada, se pueden invocarlos métodos de gestión de persistencia proporcionados por el objeto Session, para recuperar, añadir, eliminar o modificar el estado de instancias de clases persistentes. También se pueden realizar consultas. Si las operaciones de persistencia no han producido ninguna excepción, se invoca el método commit de la unidad de trabajo para confirmar los cambios realizados. En caso contrario, se realiza un rollback para deshacer los cambios producidos. Sobre un mismo objeto Session pueden crearse varias unidades de trabajo. Finalmente se cierra el objeto Session invocando su método close.

Ejemplo básico de cómo usar Hibernate con Mapeo con Anotaciones

Este ejemplo incluye los pasos fundamentales, como la configuración de Hibernate, la creación de una entidad, y las operaciones básicas de CRUD (Crear, Leer, Actualizar, Eliminar) usando Hibernate.

Pasos previos

1. **Configura tu entorno de desarrollo**: Asegúrate de tener configurado un proyecto en Java (puedes usar IntelliJ IDEA o Eclipse) con acceso a una base de datos MySQL.
2. **Agrega las dependencias de Hibernate y MySQL**: Si estás usando Maven, agrega estas dependencias en tu archivo **pom.xml**.

Dependencias en pom.xml

```
<dependencies>
    <!-- Hibernate Core -->
    <dependency>
        <groupId>org.hibernate</groupId>
        <artifactId>hibernate-core</artifactId>
        <version>5.6.5.Final</version>
    </dependency>

    <!-- MySQL Connector -->
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.28</version>
    </dependency>
</dependencies>
```

Paso 1: Configura Hibernate con hibernate.cfg.xml

Crea un archivo llamado **hibernate.cfg.xml** en la carpeta src/main/resources. Este archivo contiene la configuración para conectarse a MySQL y los mapeos de Hibernate.

```
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- Configuración de la base de datos -->
        <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/tu_base_de_datos</property>
        <property name="hibernate.connection.username">tu_usuario</property>
        <property name="hibernate.connection.password">tu_contraseña</property>

        <!-- Dialecto de MySQL -->
        <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>

        <!-- Opciones adicionales -->
        <property name="hibernate.show_sql">true</property>
        <!-- Configuración de hbm2ddl para crear y actualizar el esquema automáticamente -->
        <property name="hibernate.hbm2ddl.auto">update</property>

        <!-- Añade las clases mapeadas -->
        <mapping class="com.ejemplo.Usuario"/>
    </session-factory>
</hibernate-configuration>
```

NOTA: Asegúrate de reemplazar "tu_base_de_datos", "tu_usuario" y "tu_contraseña" con los datos de tu base de datos MySQL. Asegúrate de añadir las clases mapeadas "com.ejemplo.Usuario".

Paso 2: Crea una Entidad Java (Usuario.java)

Crea una clase de entidad simple llamada `Usuario` que represente una tabla en tu base de datos.

```
package com.ejemplo;
import javax.persistence.*;

@Entity
@Table(name = "usuarios")
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nombre")
    private String nombre;

    @Column(name = "email")
    private String email;

    // Constructores, getters y setters
    public Usuario() {}

    public Usuario(String nombre, String email) {
        this.nombre = nombre;
        this.email = email;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

Paso 3: Crea una Clase de Utilidad para Hibernate (HibernateUtil.java)

Esta clase ayuda a crear y administrar la sesión de Hibernate.

```
package com.ejemplo;


import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;


public class HibernateUtil {
    private static SessionFactory sessionFactory = buildSessionFactory();
    private static SessionFactory buildSessionFactory() {
        try {
            return new Configuration().configure("hibernate.cfg.xml").buildSessionFactory();
        } catch (Throwable ex) {
            System.err.println("Error inicializando SessionFactory: " + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }
    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
    public static void shutdown() {
        getSessionFactory().close();
    }
}
```

Paso 4: Operaciones CRUD en una Clase Principal (Main.java)

Aquí tienes un ejemplo de cómo realizar operaciones básicas de CRUD en la base de datos usando Hibernate.

```
package com.ejemplo;


import org.hibernate.Session;
import org.hibernate.Transaction;


public class Main {
    public static void main(String[] args) {
        // Crear un usuario
        Usuario usuario = new Usuario("Juan Pérez", "juan@example.com");
        saveUsuario(usuario);
        // Leer el usuario
        Usuario usuarioLeido = getUsuario(usuario.getId());
        System.out.println("Usuario leído: " + usuarioLeido.getNombre());
        // Actualizar el usuario
        usuarioLeido.setNombre("Juan Pérez Actualizado");
        updateUsuario(usuarioLeido);
        // Eliminar el usuario
        deleteUsuario(usuarioLeido.getId());
        // Cerrar la sesión de Hibernate
        HibernateUtil.shutdown();
    }


    public static void saveUsuario(Usuario usuario) {
        Transaction transaction = null;
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();
            session.save(usuario);
            transaction.commit();
            System.out.println("Usuario guardado: " + usuario.getNombre());
        } catch (Exception e) {
            if (transaction != null) transaction.rollback();
            e.printStackTrace();
        }
    }
```

```
  public static Usuario getUsuario(Long id) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            return session.get(Usuario.class, id);
        }
    }
    public static void updateUsuario(Usuario usuario) {
        Transaction transaction = null;
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();
            session.update(usuario);
            transaction.commit();
            System.out.println("Usuario actualizado: " + usuario.getNombre());
        } catch (Exception e) {
            if (transaction != null) transaction.rollback();
            e.printStackTrace();
        }
    }
    public static void deleteUsuario(Long id) {
        Transaction transaction = null;
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();
            Usuario usuario = session.get(Usuario.class, id);
            if (usuario != null) {
                session.delete(usuario);
                System.out.println("Usuario eliminado: " + usuario.getNombre());
            }
            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) transaction.rollback();
            e.printStackTrace();
        }
    }
}
```

Paso 5: Ejecuta el Proyecto

1. Asegúrate de tener la base de datos MySQL en ejecución.
2. Ejecuta la clase `Main` para realizar las operaciones CRUD.
3. Observa en la consola los mensajes de cada operación y revisa la base de datos para confirmar los cambios.

Este es un ejemplo básico, y es posible expandirlo para manejar transacciones de forma más robusta y añadir excepciones para una mejor gestión de errores en aplicaciones más complejas.